SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act,1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

END-SEMESTER PROJECT REPORT

on

MAT339 - TRANSFORMER MODELS & SEQUENCE LEARNING

Submitted By

Student Name	:	Abirami B
Register Number	:	127150003
Programme	:	M.Sc. Data Science
Academic Year	:	2026 – 2027
School	:	School of Arts, Sciences, Humanities & Education
Project Title	:	Comparative Analysis of Recurrent Neural Networks and Transformer Models for AG News Topic Classification

Internal Faculty Signature **External Faculty Signature(s)**

Contents

List of Figures 3

List of Tables 4

List of Abbreviations 5

Abstract 6

1 Introduction 7

2 Problem Statement 10

3 Objectives 12

4 Dataset Description 13

5 Methodology 17

6 Experimental Setup 27

7 Evaluation Metrics 31

8 Results and Analysis 34

9 Sample Predictions 40

10 Discussion 43

11 Limitations 46

12 Future Enhancements 48

Bibliography 49

A Source Code 52

List of Figures

Figure 4.1 Class distribution of the official training and test splits (counts verified from the notebook output). 14

Figure 4.2 Article length in words and in BERT tokens. Dashed lines mark the maximum lengths used (64 words, 128 tokens). Figure generated by the project notebook (Section 4.2). 14

Figure 5.1 Workflow of the experimental pipeline (original diagram representing the notebook implementation). 17

Figure 5.2 Structure of the shared recurrent classifier (original diagram of the notebook's RecurrentClassifier). 20

Figure 5.3 Schematic of scaled dot-product attention (original diagram; no data are plotted). 22

Figure 5.4 Fine-tuning pipeline of BERT and DistilBERT as implemented in the project (original schematic). 24

Figure 6.1 Number of parameters of the five models (logarithmic axis; values from the notebook output). 29

Figure 8.1 Test accuracy, macro F1, training time and inference time for the five models. 35

Figure 8.2 Per-class F1-score of the five models on the full test set (values from the classification reports). 36

Figure 8.3 Confusion matrices of the five models on the full 7,600-article test set (rows: true class; columns: predicted class). 37

List of Tables

Table 4.1 Dataset summary (official splits, verified in the notebook output) 13
Table 4.2 Features, data types and label encoding 13
Table 4.3 Data-quality audit of the official splits 13
Table 4.4 Article-length statistics (training set) 14
Table 4.5 Representative training articles (excerpts) 15
Table 4.6 Notebook-printed articles whose topic boundary is not obvious from the excerpt 15
Table 4.7 Derived partitions used for the reported (full-dataset) experiments 16
Table 5.1 Vocabulary built from the training split (recurrent models) 18
Table 5.2 One training article encoded for the recurrent models and for the Transformers (as printed by the notebook) 19
Table 5.3 Symbols used in the equations 19
Table 5.4 Comparison of the recurrent layers used in this project 22
Table 5.5 Summary of the five models as used in this project 24
Table 6.1 Execution environment recorded in the notebook output 27
Table 6.2 Experimental configuration by model family 27
Table 6.3 Training tensors passed to the models (full dataset) 28
Table 6.4 Parameter counts (total = trainable) 29
Table 7.1 Per-class counts and metrics for BERT derived from its confusion matrix (7,600 test articles) 32
Table 8.1 Test-set performance (full dataset, 7,600 test articles), ranked by accuracy 34
Table 8.2 Computational cost (full dataset). Per-article inference time is derived from the two-decimal total and is approximate 34
Table 8.3 Per-class precision, recall and F1-score on the full test set (1,900 articles per class) 35
Table 8.4 Per-class F1-score of LSTM (best recurrent model) and BERT, and the difference in percentage points 36
Table 8.5 Most frequent misclassifications per model (true class → predicted class, article count) 37
Table 8.6 Pairwise differences in test accuracy (first model minus second model) 37
Table 8.7 Computational cost ratios 38
Table 8.8 DistilBERT relative to BERT 38
Table 8.9 Ranking by test accuracy, macro F1 and accuracy per training second (full dataset) 38
Table 8.10 Summary of the measured results by research question (full dataset, one seed) 39
Table 9.1 Correctly classified test articles per class (out of 1,900 per class; 7,600 in total) 40
Table 9.2 Misclassified test articles by true class (out of 1,900 per class) and errors on the Business–Sci/Tech boundary 41
Table 9.3 Number of test articles predicted as each class (column totals of the confusion matrices; difference from the true count of 1,900 in brackets) 41
Table 9.4 Possible sources of error and the evidence available in the project 41
Table 10.1 Model choice by constraint, based on the full-dataset measurements 44
Table 11.1 Effect of each main limitation on the conclusions and the treatment in this report 47

List of Abbreviations

Abbreviation	Meaning
AG News	AG's corpus of news articles (benchmark dataset)
BERT	Bidirectional Encoder Representations from Transformers
CUDA	Compute Unified Device Architecture
DistilBERT	Distilled version of BERT
EDA	Exploratory data analysis
F1	Harmonic mean of precision and recall
FN / FP	False negatives / false positives
fp16 / fp32	16-bit / 32-bit floating-point precision
GPU	Graphics processing unit
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
NLP	Natural Language Processing
OOV	Out of vocabulary
RNN	Recurrent neural network
TN / TP	True negatives / true positives
WordPiece	Sub-word tokenisation algorithm used by BERT

Abstract

News organisations, aggregators and analytics platforms need to assign large streams of articles to topics automatically. This project compares five neural architectures for four-class news topic classification on the AG News benchmark [1]: three recurrent models trained from scratch (Simple RNN, LSTM and GRU) and two pretrained Transformer encoders fine-tuned for the task (BERT-base and DistilBERT-base). All models use the complete official training and test partitions. The official training split is divided into 108,000 training and 12,000 validation articles, while all 7,600 official test articles are retained for final evaluation. The models are trained for five epochs and the checkpoint with the lowest validation loss is restored before test evaluation. BERT obtained the highest test accuracy (0.9486, macro F1 0.9486), followed by DistilBERT (0.9464, 0.9464), LSTM (0.9212, 0.9209), GRU (0.9211, 0.9208) and Simple RNN (0.9121, 0.9118). DistilBERT used 61.2% of BERT's parameters and about half of its training and inference time, with about 0.21 percentage-point lower accuracy. The Transformers required substantially more computation than the recurrent models. Business and Sci/Tech remained the most frequently confused classes. The study uses one random seed, so small differences should be interpreted cautiously.

Keywords: Text classification; AG News; Recurrent neural network; LSTM; GRU; Transformer; BERT; DistilBERT; Transfer learning; Model comparison

Chapter 1

Introduction

1.1 Background

The volume of digital news published every day makes manual categorisation impractical. Automated topic classification assigns each article to a section such as World, Sports, Business or Science and Technology, which supports content organisation, personalised recommendation, search, monitoring and the construction of downstream NLP pipelines. The task is a standard instance of supervised multiclass text classification: given the text of an article, predict one label from a fixed set.

1.2 Applications of news topic classification

Topic labels are used wherever a collection of articles must be organised or routed. Typical uses include arranging articles into the sections of a news site, selecting articles for topic-specific newsletters or alerts, supporting recommendation and search by topic, and monitoring a stream of articles for subjects of interest. These uses share one requirement that motivates automatic classification: the assignment must be fast and consistent for a large and continuous stream of text. The AG News task studied in this project is a simplified version of this problem, with four coarse sections and exactly one label per article.

1.3 Text classification in Natural Language Processing

Text classification is among the most widely studied tasks in Natural Language Processing (NLP) and is used as a benchmark for new representation-learning methods. Large-scale benchmark datasets such as AG News were introduced to evaluate classifiers on sizeable corpora [1]. Because the task has well-defined labels and simple metrics, it allows architectures with very different inductive biases to be compared under common conditions.

1.4 Evolution of text representations

Early text classifiers represented a document as a sparse vector of word counts or weighted counts, which ignores word order and treats every word as an unrelated symbol. Neural approaches replaced these vectors with dense, learned vectors called embeddings, in which words used in similar contexts receive similar vectors [2, 3]. Static embeddings give each word one vector regardless of context, so a word with several meanings has a single representation. Contextual models address this by computing the representation of every word from the whole sentence: ELMo derived such representations from a bidirectional recurrent language model [4], and Transformer models such as BERT compute them with self-attention [5]. A further step is transfer learning, in which a language model is first trained on a large unlabelled corpus and then adapted to a labelled task with comparatively little data [6].

The five models in this project span this range. The three recurrent models learn their embeddings from the 108,000 training articles only, whereas BERT and DistilBERT start

from representations pretrained on large unlabelled corpora and only fine-tune them. Comparing the two groups under one pipeline therefore shows how the complete training configurations perform on the full AG News training set, although the design does not isolate pretraining from the other differences between the groups (Section 6.7).

1.5 Motivation for comparing recurrent and Transformer architectures

Recurrent neural networks (RNNs) process a text one token at a time and carry a hidden state forward [7]. Gated variants, the Long Short-Term Memory network (LSTM) [8] and the Gated Recurrent Unit (GRU) [9], were designed to ease the learning of longer-range dependencies. Transformer encoders replace recurrence with self-attention [10] and are pretrained on very large corpora before being fine-tuned, as in BERT [5] and its distilled counterpart DistilBERT [11]. Practitioners therefore face a practical question: how much accuracy does a pretrained Transformer add over a recurrent model trained from scratch on the same labelled data, and at what computational cost? This project answers that question empirically for AG News with five models implemented in one reproducible notebook.

1.6 Literature review

1.6.1 Recurrent networks and the vanishing-gradient problem

Simple recurrent networks, popularised by Elman [7], update a hidden state from the current input and the previous state. Training such networks by back-propagation through time is difficult because gradients can shrink or grow exponentially with sequence length, which makes long-range dependencies hard to learn [12, 13]. Gradient clipping, which the project's training loop also applies, is a standard remedy for exploding gradients [13]. Bidirectional recurrent networks read a sequence in both directions and combine the two hidden states [14].

1.6.2 LSTM and GRU

The LSTM introduced a memory cell with multiplicative gates to preserve information over many steps [8]; the forget gate used in modern implementations was added later [15]. The GRU, proposed for sequence-to-sequence learning, merges the cell and hidden state and uses two gates, giving a smaller parameter count than an LSTM of the same width [9].

1.6.3 Attention and Transformers

Attention was first used to let a recurrent encoder–decoder look back over its input [16]. The Transformer dispensed with recurrence and convolution and relied on scaled dot-product self-attention, allowing every position to attend to every other position and enabling parallel computation over a sequence [10].

1.6.4 BERT and DistilBERT

BERT pretrains a deep bidirectional Transformer encoder on unlabelled text with a masked-language-modelling objective and can be fine-tuned for classification by adding an output layer [5]. It uses WordPiece sub-word tokenisation [17]. The base configuration has 12 layers and about 110 million parameters [5]. DistilBERT applies knowledge distillation [18] during pretraining to obtain a model with six layers that its authors report to be 40% smaller and 60% faster than BERT while retaining about 97% of its language-understanding performance on the GLUE benchmark [11].

1.6.5 AG News and evaluation of text classifiers

AG News was constructed from a large collection of news articles by selecting the four largest classes and using the title and description fields, yielding 30,000 training and 1,900 test articles per class [1]. Classifiers on such balanced benchmarks are commonly summarised with accuracy and with macro-averaged precision, recall and F1-score, the latter weighting every class equally [19].

The choice of performance measure influences how classifiers are ranked, and systematic analyses of measures for classification tasks are available [20]. For a paired comparison of two classifiers evaluated on one shared test set, McNemar's test has been recommended [21], and the use of significance tests in NLP has been surveyed [22]. None of these tests is applied in this project (Section 8.7); they motivate the future work described in Chapter 12.

1.6.6 Optimisation and regularisation of neural classifiers

Neural classifiers are trained by gradient descent on a loss function. The Adam optimiser adapts the step size of every parameter using running estimates of the first and second moments of the gradient [23], and AdamW decouples the weight-decay term from this adaptive update [24]; AdamW is the optimiser used here. Dropout randomly sets activations to zero during training to reduce co-adaptation of units [25]. Deep Transformer stacks are stabilised by residual connections [26] and layer normalisation [27], and mixed-precision arithmetic reduces memory use and training time [28]. Chapter 5 describes how each of these components is used in the implemented pipeline.

1.7 Research gap and contribution of this project

This project does not claim a new method. Its contribution is a controlled and fully reproducible comparison of five architectures on one benchmark, in which predictive performance, training time, inference time and parameter count are measured by the same code path, and in which the confusion patterns of every model are inspected. The study states openly which factors are *not* controlled (pretraining, tokenisation, learning rate and numeric precision differ between the recurrent and Transformer families), so that the results are interpreted as a comparison of practical pipelines rather than of architectures in isolation.

1.8 Scope

The scope is limited to single-label, four-class topic classification of English news text from AG News, using the five models listed above, a single random seed, and the hyperparameters fixed in the notebook. Other tasks such as sentiment analysis, summarisation or fake-news detection are outside the scope.

1.9 Organisation of the report

Chapter 2 formulates the problem and the research questions, and Chapter 3 lists the objectives. Chapter 4 describes the dataset and its exploratory analysis. Chapter 5 presents the preprocessing pipeline and the model architectures. Chapter 6 documents the experimental setup and Chapter 7 the evaluation metrics. Chapter 8 reports the results, Chapter 9 explains the status of the sample-prediction analysis, and Chapter 10 discusses the findings and gives the conclusion. Chapters 11 and 12 describe limitations and future enhancements. The bibliography and the source-code appendix follow.

Chapter 2

Problem Statement

2.1 Formal problem statement

Let x be a news article represented by its title and description joined into a single string, and let $y \in$ {World, Sports, Business, Sci/Tech} be its topic label, encoded as the integers 0 to 3. Given a training set of labelled articles, the task is to learn a function f that maps an unseen article to its most probable topic. Performance is measured on held-out articles that are never used for training, hyperparameter choice or checkpoint selection.

2.2 Input and output specification

- **Input.** One English news article represented by a single string that joins its title and description (mean 37.8 words, maximum 177 words in the training set).
- **Output.** A probability distribution over the four classes World, Sports, Business and Sci/Tech (label identifiers 0 to 3); the predicted topic is the class with the highest probability.
- **Supervision.** Only the article's label is used for learning; no additional metadata, source information or external text is supplied to any model.
- **Partitions. 108,000 training articles, 12,000 validation articles and all 7,600 official test articles in the experiments reported here (Table 4.7).**

2.3 Characteristics that make the task difficult

Although AG News is a balanced and well-curated benchmark, the following characteristics, observed in the project's data audit and examples, make accurate classification non-trivial.

- **Topic boundaries.** Some articles legitimately fit two classes. The notebook's printed examples include an article about a chip-maker's market outlook labelled Sci/Tech and a football-related headline labelled Sci/Tech (Table 4.6).
- **Short texts.** An article has a median of 37 words, so a model has little context from which to infer the topic.
- **Surface noise.** HTML-entity residue such as `#39;` and `amp;` remains in the text and is turned into spurious tokens by the word-level tokeniser.
- **Limited vocabulary for models trained from scratch. The recurrent models use a capped 20,000-word training vocabulary, and 2.87% of validation words are outside it.**

2.4 Aim

The aim of the project is to compare, under one reproducible experimental pipeline, how recurrent neural networks trained from scratch and pretrained Transformer models fine-tuned for the task perform on AG News topic classification, and what computational cost accompanies any difference in predictive quality.

2.5 Research questions

The following research questions are investigated. They are questions to be answered by the measured evidence, not findings assumed in advance.

1. **RQ1.** Do the pretrained Transformers (BERT, DistilBERT) classify AG News more accurately than the recurrent models trained from scratch?
2. **RQ2.** Among Simple RNN, LSTM and GRU, does the gating mechanism help compared with a plain RNN?
3. **RQ3.** How much additional computational cost (training time, inference time, parameters) do the Transformers require, and what accuracy do they gain for it?
4. **RQ4.** Does DistilBERT offer a better accuracy-versus-cost trade-off than BERT?
5. **RQ5.** Which topic categories are most frequently confused by the evaluated models?

2.6 Assumptions and constraints

- Each article has exactly one label, and the label set is fixed to the four AG News classes.
- The official test articles are never used for training or model selection; the validation split is derived from the training data.
- The experiments reported here use the complete official AG News training and test splits, a single random seed (42) and fixed hyperparameters.
- All five models are trained and timed on one Tesla T4 GPU, and no external data or additional labelled resources are used.

Chapter 3

Objectives

The objectives of the project are:

1. To develop a reproducible pipeline for AG News topic classification using the complete official training and test datasets.

2. To divide the official training dataset into training and validation sets using stratified sampling while keeping the official test set unchanged.

3. To implement and train three recurrent neural network models: Simple RNN, LSTM and GRU.

4. To fine-tune two pretrained Transformer models, BERT and DistilBERT, for news topic classification.

5. To compare all five models using the same evaluation metrics: accuracy, macro precision, macro recall and macro F1-score.

6. To analyse the computational requirements of each model based on training time, inference time and parameter count.

7. To identify frequently misclassified news categories using per-class classification reports and confusion matrices.

8. To compare the performance and computational efficiency of recurrent and Transformer models, document the limitations of the experiments and suggest possible future enhancements.

Chapter 4

Dataset Description

4.1 Source and structure

The project uses the AG News topic-classification dataset, loaded with the Hugging Face `datasets` library [29] from the repository `fancyzhx/ag_news` [30]. The benchmark was introduced in [1]. The dataset has an official training split of 120,000 articles and an official test split of 7,600 articles. Each record has two fields: `text`, the article title and description joined in one string, and `label`, an integer class identifier (Tables 4.1 and 4.2).

Table 4.1: Dataset summary (official splits, verified in the notebook output)

Split	Articles	Per class	Classes	Share of classes
Official training	120,000	30,000	4	25.0% each
Official test	7,600	1,900	4	25.0% each

Table 4.2: Features, data types and label encoding

Field	Pandas dtype	Dataset feature type	Description
text	object	Value('string')	Title and description of the article
label	int64	ClassLabel (4 names)	0 = World, 1 = Sports, 2 = Business, 3 = Sci/Tech

The notebook asserts that the label names returned by the dataset are exactly `['World', 'Sports', 'Business', 'Sci/Tech']` and prints one example per label identifier to confirm the mapping.

4.2 Data-quality audit

Section 4.4 of the notebook audits both official splits. The results are summarised in Table 4.3. No cleaning action was needed for the official data; the cleaning function in the pipeline (drop rows with missing text or label, strip whitespace, remove empty texts) therefore left the split sizes unchanged at 120,000 and 7,600.

Table 4.3: Data-quality audit of the official splits

Check	Training set	Test set
Missing text	0	0
Missing label	0	0
Empty text	0	0
Exact duplicate texts	0	0
Texts with more than one label	0	not computed
Texts present in both training and test sets	0	(same check)
Rows after cleaning	120,000	7,600

4.3 Class distribution

Both official splits are perfectly balanced: each of the four classes contains 30,000 training articles and 1,900 test articles (Figure 4.1). Because the classes are balanced, accuracy is a meaningful headline metric; macro-averaged metrics are still reported because they expose per-class behaviour.

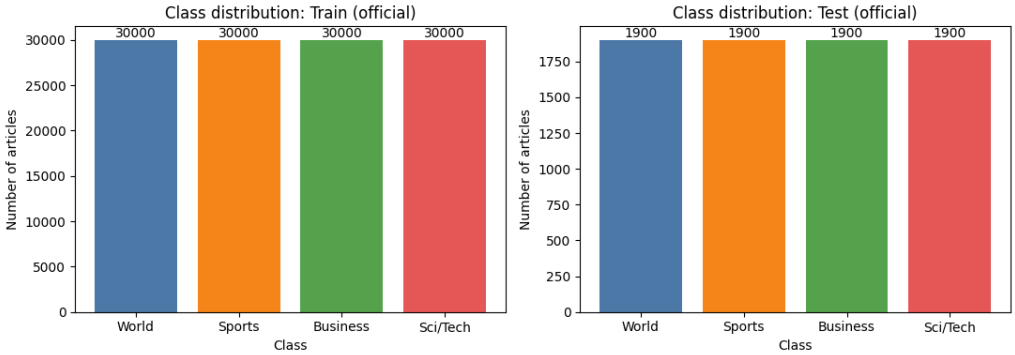


Figure 4.1: Class distribution of the official training and test splits (counts verified from the notebook output).

4.4 Text-length analysis

Article length was measured in whitespace-separated words (all 120,000 training articles) and in BERT WordPiece tokens including the special tokens (a random sample of 20,000 training articles, seed 42). Table 4.4 gives the statistics and Figure 4.2 the distributions produced by the notebook. Articles are short: the median is 37 words and 50 tokens. Only 1.83% of training articles exceed the recurrent models' 64-word limit and 0.74% of the sampled articles exceed the Transformers' 128-token limit, so truncation affects few articles. Token counts exceed word counts because WordPiece splits rare words into pieces. Mean article length is similar across classes (Business 37.5, Sci/Tech 37.2, Sports 37.8, World 38.9 words), so length is unlikely to be a strong class signal.

Table 4.4: Article-length statistics (training set)

Unit (sample)	Count	Mean	Std	Min	25%	50%	75%	Max
Words (all training)	120,000	37.8	10.1	8	32	37	43	177
BERT tokens (20k sample)	20,000	53.1	19.2	18	43	50	59	379

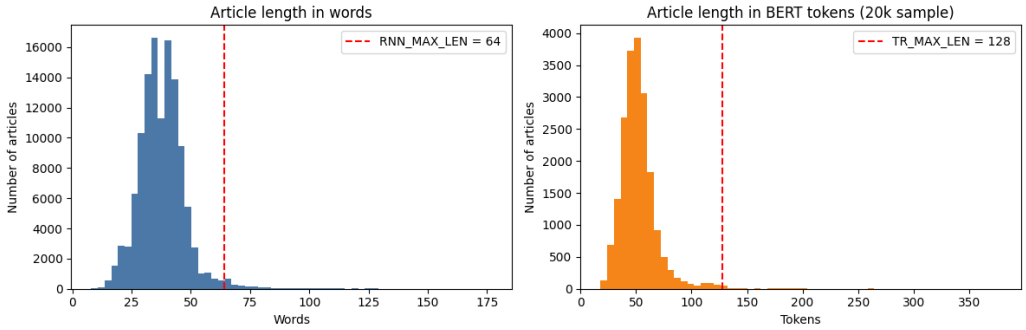


Figure 4.2: Article length in words and in BERT tokens. Dashed lines mark the maximum lengths used (64 words, 128 tokens). Figure generated by the project notebook (Section 4.2).

4.5 Representative examples and data-quality observations

Table 4.5 shows one training article per class, shortened for display. The examples illustrate the style of the data: a headline followed by one or two sentences, often with agency tags. Two observations follow from the notebook's printed examples and vocabulary output. First, the text contains HTML-entity residue such as `#39;` for an apostrophe and `amp;` for an ampersand; the notebook does not clean these, and the recurrent models' tokeniser turns `#39;s` into the tokens `39` and `s`, which both appear among the ten most frequent words of the training vocabulary. Second, some articles legitimately fit two classes; for example, an article on a chip-maker's market outlook is labelled Sci/Tech although its content concerns industry growth. These observations are *hypotheses* about potential noise and label overlap, not measured effects.

Table 4.5: Representative training articles (excerpts)

Class	Excerpt
World	Explosion Rocks Baghdad Neighborhood. BAGHDAD, Iraq, August 24 -- A car bomb exploded near the gate of a US-funded Iraqi television network in Baghdad on Tuesday, killing at least two people ...
Sports	Second Andre win in a row boosts US gold medal hopes. Andre Dirrell ... assured the United States of at least two boxing medals Wednesday ...
Business	US house sales fall in July. Sales of non-new houses in the US fell last month but still exceeded analyst forecasts.
Sci/Tech	Gartner optimistic about chip numbers. But that optimism isn #39;t matched by Infineon, which said that while the market worldwide remains buoyant ...

4.6 Observed label ambiguity

Table 4.6 lists three articles printed by the notebook for which the topic boundary is not obvious from the headline or excerpt. They are observations about labelled examples, not evidence about any model's errors; the printed excerpts are too short to establish why each label was assigned. They are included because the dominant confusion in the results (Chapter 8) is between Business and Sci/Tech.

Table 4.6: Notebook-printed articles whose topic boundary is not obvious from the excerpt

Assigned class	Excerpt
Sci/Tech	Gartner optimistic about chip numbers. But that optimism isn #39;t matched by Infineon, which said that while the market worldwide remains buoyant ...
Sci/Tech	'Madden,' 'ESPN' Football Score in Different Ways (Reuters) Reuters - Was absenteeism a little high on Tuesday ...
Business	DJ to acquire MarketWatch. Dow Jones amp; Co., publisher of the Wall Street Journal, says it has agreed to buy online news site MarketWatch Inc. in a $519 million deal that widens its Internet audience.

4.7 Implications of the dataset characteristics for modelling

- **Balanced classes.** Because every class has the same number of articles, accuracy is a meaningful headline metric and, on a balanced test set, macro recall equals accuracy (Chapter 7).
- **Short articles.** A maximum length of 64 words truncates 1.83% of training articles for the recurrent models, and 128 tokens truncates 0.74% of the sampled articles for the Transformers, so truncation is unlikely to dominate the results.

- **Surface noise.** HTML-entity residue is not cleaned. It affects the word-level vocabulary (two of the ten most frequent entries are `39` and `s`) and is split into sub-words by the Transformer tokenisers.
- **Leakage checks.** The audit found no duplicate texts and no text shared between the training and test sets, so exact duplicates cannot inflate test accuracy. Near-duplicate articles were not examined.
- **Temporal and source coverage.** The time span and sources of the articles were not analysed in the project, so conclusions apply to the benchmark as provided.

4.8 Official splits versus derived partitions

It is important to distinguish the dataset's official training and test splits from the derived partitions used in the experiments. The official test set is never used for training, tuning or model selection. The complete 120,000-article official training split is divided into 108,000 training and 12,000 validation articles by a stratified random split with test_size = 0.10 and random_state = 42. The official test set of 7,600 articles is retained without subsampling (Table 4.7).

Table 4.7: Derived partitions used for the reported (full-dataset) experiments

Partition	Articles	World	Sports	Business	Sci/Tech	Share of 127,600
Training	108,000	27,000	27,000	27,000	27,000	84.64%
Validation	12,000	3,000	3,000	3,000	3,000	9.40%
Test (official)	7,600	1,900	1,900	1,900	1,900	5.96%
Total	127,600					100%

Chapter 5

Methodology

5.1 Overall workflow

Figure 5.1 shows the workflow implemented in the notebook. The complete official training and test splits are loaded and audited, the training split is divided into training and validation parts, and the data are encoded in two different ways: word-level encoding for the recurrent models and WordPiece encoding for the Transformers. Every model is trained with the same five-epoch budget, the best checkpoint on the validation split is restored, and the full official test set is evaluated once.

> **Experimental status. All results in this report come from the full official AG News dataset: 108,000 training, 12,000 validation and 7,600 test articles, using one random seed.**

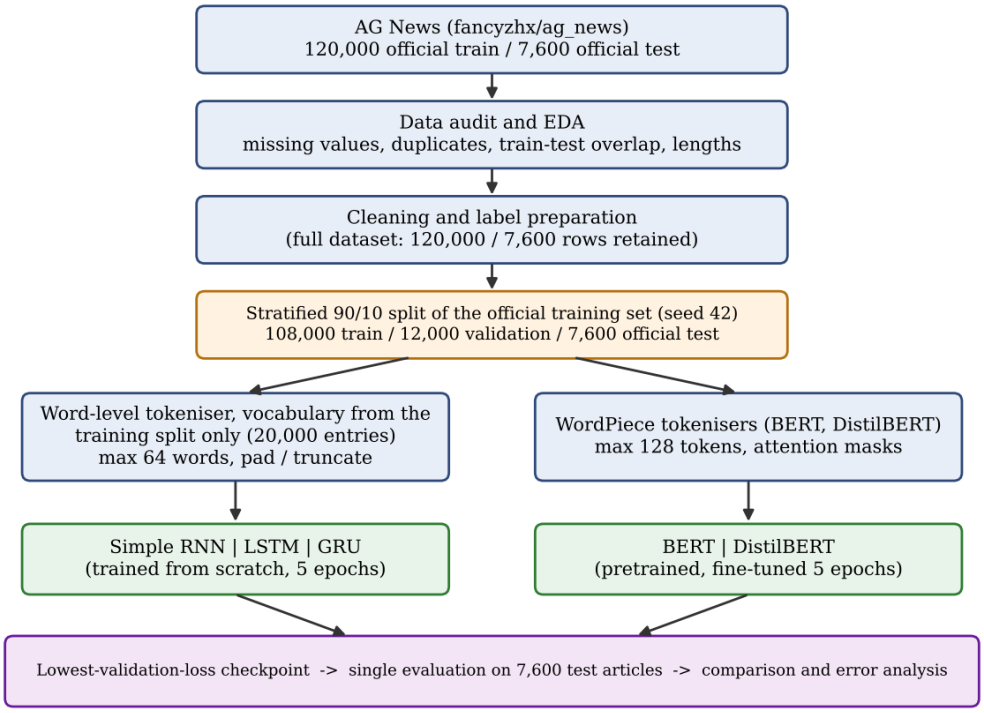


Figure 5.1: Workflow of the experimental pipeline (original diagram representing the notebook implementation).

The remainder of the chapter follows the workflow in order. Section 5.2 describes the data preparation in detail, Section 5.3 explains the five architectures together with the background needed to understand them, and Section 5.4 describes how every model is trained and how the final model is chosen.

5.2 Preprocessing

5.2.1 Cleaning, label preparation and splitting

The cleaning function drops rows with a missing text or label, strips surrounding whitespace, removes empty texts and casts labels to integers. The complete official training split of 120,000 articles is divided into 108,000 training and 12,000 validation articles by a stratified split (10% validation, seed 42). The official test set of 7,600 articles is retained in full. The partitions are therefore stratified random splits, not simple random splits, and all five models use exactly the same partitions.

5.2.2 Word-level encoding for the recurrent models

Text is lower-cased and split with the regular expression [a-z0-9]+(?:'[a-z]+)?. A vocabulary is built from the 108,000-article training split only, so that no information from the validation or test sets leaks into it. Words occurring fewer than three times are discarded and at most 20,000 entries are allowed, including two reserved tokens, <pad> (index 0) and <unk> (index 1). The vocabulary reaches its 20,000-entry cap. Words not in the vocabulary map to <unk>; the out-of-vocabulary rate on validation words is 2.87%. Each article is truncated to 64 words and right-padded with the padding index, and its true length is stored so that packed sequences can skip the padding. An article whose tokenisation is empty is replaced by a single <unk> token. The training tensors have shape (108,000 × 64).

Table 5.1 summarises the vocabulary that results from these settings. The vocabulary reaches its 20,000-entry cap, so the cap rather than the minimum-frequency threshold determines its size: at least 19,998 distinct words occur three or more times in the training split (the exact number of qualifying words is not printed by the notebook). Padding index 0 and unknown index 1 are reserved, which means that a padded position never contributes to the pooled features, because the model masks positions beyond each article's true length.

Table 5.1: Vocabulary built from the training split (recurrent models)

Property	Value
Built from	Training split only (108,000 articles)
Tokeniser	Lower-case regular expression for words and simple contractions
Minimum word frequency	3
Maximum vocabulary size	20,000 (reached)
Actual vocabulary size	20,000 entries, including <pad> (0) and <unk> (1)
Ten most frequent entries	the, to, a, of, in, and, on, for, 39, s
Out-of-vocabulary rate (validation words)	2.87%
Maximum sequence length	64 words (right-padded, truncated)

5.2.3 Sub-word encoding for the Transformers

BERT and DistilBERT use their own pretrained uncased WordPiece tokenisers [17]. Each article is encoded with the special [CLS] and [SEP] tokens, truncated and padded to 128 tokens, and accompanied by an attention mask that marks real tokens, producing tensors of shape (108,000 × 128) for the input identifiers and the mask. Unlike the word-level tokeniser, WordPiece keeps punctuation marks such as hyphens, full stops and commas as separate tokens (Table 5.2), and it splits words that are absent from its vocabulary into sub-word pieces marked with ## instead of mapping them to an unknown token. This

difference in representation, together with the different maximum lengths (64 words versus 128 tokens), is one of the factors that is not held constant between the model families.

Both Transformers share the same maximum length of 128 tokens but use their own vocabularies. Each article becomes two integer vectors of length 128, the token identifiers and the attention mask, and the whole training split is stored as tensors of shape (108,000 × 128) together with the label vector.

5.2.4 Worked tokenisation example

Table 5.2 follows the first article of the 108,000-article training split through both encoders, using the values printed by the notebook. The word-level tokeniser lower-cases *ATHENS* and discards the hyphen, the full stop and the comma, whereas the WordPiece tokeniser adds the special token [CLS] at the start and keeps each punctuation mark as a token of its own. The word *seconds* occurs twice and receives the same identifier (1780) both times, which illustrates that the recurrent models start from one fixed vector per word, whereas the Transformers combine the token vector with context in every layer. The article is 53 words long after word-level tokenisation, well within the 64-word limit.

Table 5.2: One training article encoded for the recurrent models and for the Transformers (as printed by the notebook)

Stage	Output (first elements)
Original text	10 seconds that change everything ATHENS - Ten seconds. Barely time enough to tie a shoe, wash a glass, get the paper of ...
Word tokens	10, seconds, that, change, everything, athens, ten, seconds, barely, time, enough, to
Word identifiers	132, 1780, 12, 675, 2617, 373, 1843, 1780, 3702, 82, 856, 3 (true length 53 words)
WordPiece tokens	[CLS], 10, seconds, that, change, everything, athens, -, ten, seconds, ., barely, time, enough, to, tie, a, shoe, ,, wash ...

5.2.5 Batching

Encoded tensors are wrapped in PyTorch `TensorDataset` and `DataLoader` objects. Training batches are shuffled with a generator seeded with 42; validation and test data are served unshuffled in batches of 128 for all models.

5.3 Model architectures

5.3.1 Notation

Table 5.3 collects the symbols used in the equations of this chapter and of Chapter 7.

Table 5.3: Symbols used in the equations

Symbol	Meaning
x_t, h_t, c_t	Input vector, hidden state and cell state at position t
W, U, b	Input weight matrix, recurrent weight matrix and bias vector
σ, tanh, $\odot$	Logistic sigmoid, hyperbolic tangent, element-wise product
T, d	Sequence length; hidden size per direction (128)
Q, K, V, d_k	Query, key and value matrices; key dimension
s_k, $p(y = k \mid x)$	Score (logit) and predicted probability of class k
N, L	Number of training articles; cross-entropy loss
TP_k, FP_k, FN_k, TN_k	True positives, false positives, false negatives, true negatives of class k

Symbol	Meaning
θ, η, λ	Parameters, learning rate, weight-decay coefficient

5.3.2 Embeddings and sequence representation

A neural network cannot process words directly, so each token identifier is mapped to a dense vector by an embedding table, a matrix with one row per vocabulary entry. In the recurrent models this table has 20,000 rows and 128 columns and is initialised randomly and learned together with the rest of the network; its 2,560,000 values form the largest part of every recurrent model (Table 6.4). The Transformers also begin with an embedding layer, but its values, like those of all later layers, come from pretraining. In both families the output of the embedding layer is a matrix with one vector per token, which the later layers transform into a representation of the whole article.

5.3.3 Simple RNN

A recurrent network maintains a hidden state h_t that summarises the tokens read so far. For input vector x_t, the Simple RNN computes Equation (5.1), where W_{xh} and W_{hh} are weight matrices, b_h is a bias and tanh is the hyperbolic tangent.

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \tag{5.1}$$

Because the same weights are applied at every step and gradients are propagated through a product of Jacobians, gradients can vanish or explode over long sequences [12, 13]; this limits how much information from early tokens survives.

All three recurrent models in this project share one class, RecurrentClassifier (Figure 5.2), and differ only in the type of recurrent layer. The input word identifiers are embedded in 128 dimensions (a randomly initialised table with 20,000 rows, the padding row fixed at zero), regularised with dropout [25] (p = 0.5), and passed through a bidirectional recurrent layer with 128 hidden units per direction, so each position receives context from both sides [14]. For each article the forward and backward states are concatenated as in Equation (5.2), giving 256 values per position. Instead of using only the last state, the model applies mean pooling and max pooling over all non-padded positions and concatenates them (Equation (5.3)), which gives a 512-dimensional vector. After a second dropout (p = 0.5), a linear layer maps the vector to four class scores.

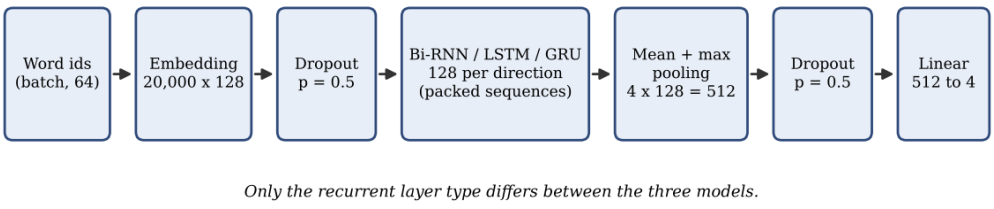


Figure 5.2: Structure of the shared recurrent classifier (original diagram of the notebook's `RecurrentClassifier`).

$$g_t = \left[\overrightarrow{h_t}\,;\,\overleftarrow{h_t}\right] \in \mathbb{R}^{2d} \tag{5.2}$$

$$z = \left[\frac{1}{T} \sum_{t=1}^{T} g_t \; ; \; \max_{1 \le t \le T} g_t \right] \tag{5.3}$$

In Equations (5.2) and (5.3), $d = 128$ is the hidden size per direction, T is the true length of the article, the arrow notation distinguishes the forward and backward states, and the maximum is taken element-wise.

5.3.4 Training recurrent networks: gradient flow

Recurrent networks are trained by back-propagation through time, which unrolls the network over the sequence and applies the chain rule across the time steps [31, 32]; textbook treatments are given in [33]. For the Simple RNN the gradient of the loss with respect to an early hidden state is a product of one Jacobian matrix per time step (Equation (5.4)).

$$\frac{\partial \mathcal{L}}{\partial h_k} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{t=k}^{T-1} \frac{\partial h_{t+1}}{\partial h_t}, \qquad \frac{\partial h_{t+1}}{\partial h_t} = \operatorname{diag}\left(1 - h_{t+1}^2\right) W_{hh} \tag{5.4}$$

Each factor contains the recurrent weight matrix W_{hh} and the derivative of the tanh function, which is at most one. When the factors are generally smaller than one in norm, the product shrinks exponentially with the distance $T - k$, so the influence of distant tokens on the loss vanishes; when they are larger than one, the product can grow and training becomes unstable [12, 13]. The gated models were designed to improve this gradient flow, and gradient clipping (Section 5.4.3) limits the instability from large gradients. In this project the articles are short (median 37 words) and the recurrent layer is followed by pooling over all positions, so the shortest paths from a token to the classifier are not long; whether the gating matters in this regime is one of the questions answered empirically in Chapter 8.

5.3.5 Long Short-Term Memory

The LSTM adds a cell state c_{t} that is modified by gates [8, 15]. The forget gate f_{t} decides how much of the previous cell state to keep, the input gate i_{t} decides how much new candidate content to write, and the output gate o_{t} decides how much of the cell to expose as the hidden state (Equations (5.5) to (5.7)).

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \qquad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \tag{5.5}$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \qquad \tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \tag{5.6}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t) \tag{5.7}$$

Here σ is the logistic sigmoid, $\odot$ denotes element-wise multiplication, W and U are input and recurrent weight matrices and b are biases. The additive update of the cell state creates a path along which gradients can flow more easily than through the repeated squashing of a Simple RNN. In this project the LSTM layer replaces the RNN layer in `RecurrentClassifier` with identical dimensions.

5.3.6 Gated Recurrent Unit

The GRU uses an update gate z_{t} and a reset gate r_{t} and has no separate cell state [9] (Equations (5.8) and (5.9)).

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \quad r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \tag{5.8}$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{5.9}$$

The reset gate controls how much of the previous state is used to form the candidate state, and the update gate interpolates between the previous state and the candidate. A GRU has three sets of weights per direction instead of the LSTM's four, so with identical dimensions it has fewer parameters (2,760,196 versus 2,826,244 in this project) and requires fewer operations per step.

5.3.7 Comparison of the recurrent cells

Table 5.4 compares the three recurrent layers. The parameter counts are those measured in this project for one bidirectional layer with 128 input and 128 hidden units; the LSTM has four blocks of weights per direction and the GRU three, which accounts for the ordering of the model sizes.

Table 5.4: Comparison of the recurrent layers used in this project

Property	**Simple RNN**	**LSTM**	**GRU**
Gates	None	Forget, input, output	Update, reset
State carried between steps	Hidden state	Hidden state and cell state	Hidden state
Weight blocks per direction	1	4	3
Parameters in the bidirectional layer	66,048	264,192	198,144
Total model parameters	2,628,100	2,826,244	2,760,196

5.3.8 From recurrence to self-attention

A recurrent model passes information from early to late tokens through a chain of state updates, and its computation over one article is sequential in time. Self-attention connects every pair of positions directly and processes all positions of a sequence in parallel within a layer [10]. The price is that attention compares every position with every other, so its cost grows with the square of the sequence length; with a maximum of 128 tokens this cost is modest here. Figure 5.3 illustrates the computation of scaled dot-product attention that Equation (5.10) defines.

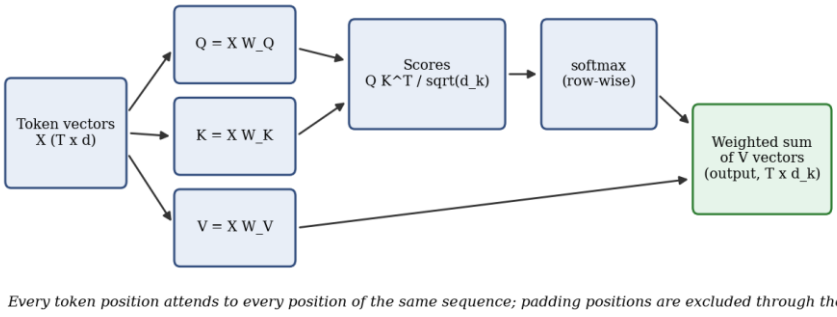


Figure 5.3: Schematic of scaled dot-product attention (original diagram; no data are plotted).

Attention itself does not depend on the order of the tokens, so position information is added to the input embeddings: the original Transformer used fixed sinusoidal encodings [10], and BERT learns a position embedding for every position [5].

5.3.9 BERT

BERT is a stack of Transformer encoder layers. Each layer applies multi-head self-attention, in which every token representation is updated as a weighted combination of all token representations in the sequence. For queries Q, keys K and values V of key dimension d_k, scaled dot-product attention is given by Equation (5.10) [10].

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{5.10}$$

Because every token can attend to every other token in both directions, each token's representation is *contextual*: the word *bank* receives different vectors in different sentences. BERT is pretrained on large unlabelled corpora and fine-tuned for a downstream task [5]. This project uses `bert-base-uncased` (12 layers, about 110 million parameters; 109,485,316 counted in the notebook) through `BertForSequenceClassification`, which adds a new, randomly initialised four-way classification layer on top of the `[CLS]` representation. A classifier dropout of 0.2 is applied before this layer. All parameters, pretrained and new, are updated during fine-tuning.

5.3.10 Multi-head attention, feed-forward layers and normalisation

A Transformer encoder layer contains two sub-layers. The first is multi-head attention, which runs several attention computations in parallel on different learned projections of the input and concatenates their results (Equation (5.11)). The second is a position-wise feed-forward network applied to every token separately. Around each sub-layer the layer adds a residual connection [26] and applies layer normalisation [27] (Equations (5.12) and (5.13)). In these equations h is the number of heads, W^Q, W^K, W^V and W^O are learned projection matrices, φ is a non-linearity (ReLU in the original Transformer [10], GELU in BERT [5]), and μ and σ^2 are the mean and variance of the layer input over its features.

$$\mathrm{MultiHead}(X) = [\mathrm{head}_1; \ldots; \mathrm{head}_h]W^O, \qquad \mathrm{head}_i = \mathrm{Attention}(XW_i^Q, XW_i^K, XW_i^V) \tag{5.11}$$

$$\mathrm{FFN}(x) = \phi(xW_1 + b_1)W_2 + b_2, \qquad y = \mathrm{LayerNorm}(x + \mathrm{Sublayer}(x)) \tag{5.12}$$

$$\mathrm{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta \tag{5.13}$$

BERT-base stacks 12 such layers with hidden size 768 and 12 attention heads [5]. Residual connections let the signal and the gradient bypass a sub-layer, which, together with normalisation, makes deep stacks trainable.

5.3.11 Input representation and pretraining

BERT represents the input as a sequence of WordPiece tokens that begins with the special `[CLS]` token and ends with `[SEP]`. The embedding of every position is the sum of a token embedding, a segment embedding and a position embedding [5]; for a single-sentence input such as a news article all segment identifiers are the same. The model is pretrained with two unsupervised objectives, predicting randomly masked tokens and predicting whether one sentence follows another [5], on large unlabelled corpora. The representation of the `[CLS]` position is designed to summarise the whole input and is what a classification head reads.

5.3.12 Fine-tuning for sequence classification

Fine-tuning adapts the pretrained encoder to the labelled task. A new classification head is attached to the encoder output, the logits are compared with the labels by the cross-entropy loss, and all parameters, pretrained and new, are updated with a small learning rate (2e-5 here). Figure 5.4 shows the pipeline as implemented for both Transformers.

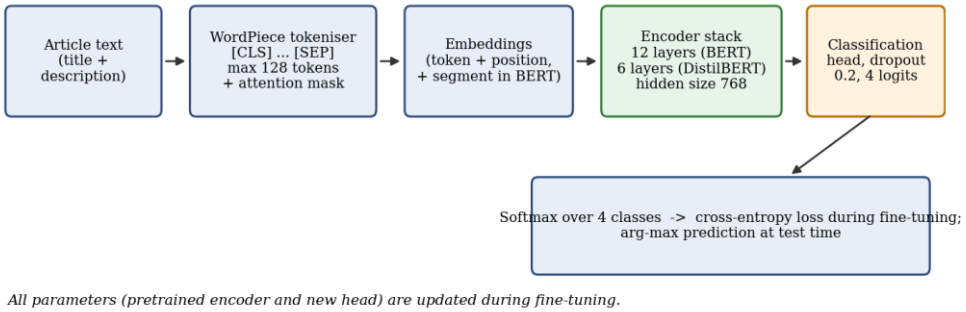


Figure 5.4: Fine-tuning pipeline of BERT and DistilBERT as implemented in the project (original schematic).

5.3.13 DistilBERT

DistilBERT is a smaller Transformer obtained by knowledge distillation, in which a compact student is trained to reproduce the behaviour of a larger teacher [18]. Its training combines a masked-language-modelling loss, a distillation loss and a cosine embedding loss (Equation (5.14)), and its architecture keeps the BERT design but uses six layers instead of twelve [11]. The expected trade-off is lower memory use and faster training and inference at the cost of a small loss in accuracy. The project fine-tunes `distilbert-base-uncased` through `DistilBertForSequenceClassification` (66,956,548 parameters) with the same data, learning rate, batch size, schedule and classifier dropout (0.2) as BERT.

$$\mathcal{L}_{\text{distil}} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{KD}} + \mathcal{L}_{\cos} \tag{5.14}$$

According to its authors, DistilBERT is initialised from the teacher by taking one layer out of two, removes the token-type embeddings and the pooler, and is trained on the same kind of corpus as BERT [11]. After this distillation the model is used exactly like BERT: it receives the same type of input and is fine-tuned with a new classification head. The practical consequence in this project is a smaller model (66,956,548 versus 109,485,316 parameters) that was trained and evaluated under identical settings.

5.3.14 Summary of the five architectures

Table 5.5 summarises how the five models differ in the properties that matter for the comparison.

Table 5.5: Summary of the five models as used in this project

Model	Token processing	Initial weights	Tokeniser	Max. length	Parameters
Simple RNN	Sequential, bidirectional	Random	Word-level	64 words	2,628,100
LSTM	Sequential, gated, bidirectional	Random	Word-level	64 words	2,826,244

Model	Token processing	Initial weights	Tokeniser	Max. length	Parameters
GRU	Sequential, gated, bidirectional	Random	Word-level	64 words	2,760,196
BERT	Parallel self-attention, 12 layers	Pretrained	WordPiece	128 tokens	109,485,316
DistilBERT	Parallel self-attention, 6 layers	Pretrained (distilled)	WordPiece	128 tokens	66,956,548

5.3.15 Classification output and loss

For all five models the final layer produces four scores $s_1,\ldots,s_4$ (logits). The softmax function converts them into class probabilities (Equation (5.15)), and the predicted class is the one with the highest score. Training minimises the cross-entropy loss over the N training articles (Equation (5.16)); no label smoothing is used.

$$p(y=k \mid x)=\frac{\exp(s_k)}{\sum_{j=1}^{4}\exp(s_j)} \tag{5.15}$$

$$\mathcal{L}=-\frac{1}{N}\sum_{n=1}^{N}\log p(y=y_n \mid x_n) \tag{5.16}$$

5.4 Training procedure

All models are trained by the same function. The seed is reset before each model is built, the AdamW optimiser [24] with decoupled weight decay of 0.01 updates all parameters, and the cross-entropy loss is minimised for five epochs. The learning rate follows a linear schedule with warm-up over the first 10% of steps followed by linear decay to zero. Gradients are clipped to a norm of 1.0. The Transformers are trained with automatic mixed precision (fp16 with gradient scaling) [28]; the recurrent models are trained in fp32. After every epoch the model is evaluated on the validation split, and the weights of the best epoch are stored (Section 5.4.5). After training, the stored weights are loaded and evaluated on the test set. The test set is not used for tuning, early stopping or checkpoint selection. If the GPU runs out of memory, the training function halves the batch size and retries; no such event is recorded in the saved outputs.

5.4.1 Optimiser and weight decay

The AdamW optimiser keeps exponential moving averages m_t and v_t of the gradient and of its square, corrects their start-up bias, and updates each parameter with a step that is normalised by the root of the bias-corrected second-moment estimate [23]. AdamW applies the weight decay directly to the parameters instead of adding it to the gradient [24], as in Equation (5.17), where η_t is the learning rate at step t and λ is the weight-decay coefficient (0.01 for all five models). In the notebook the weight decay is applied to all parameters, with no exclusion of bias or normalisation parameters.

$$\theta_t=\theta_{t-1}-\eta_t\left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\varepsilon}+\lambda\,\theta_{t-1}\right) \tag{5.17}$$

5.4.2 Learning-rate schedule

The learning rate rises linearly from zero to its maximum ηmax during the first Tw steps, which are the first 10% of all steps, and then decreases linearly to zero (Equation (5.18)).

With the training set of 108,000 articles, the recurrent models make 1,688 steps per epoch (batch size 64), or 8,440 steps in total over five epochs. The Transformers make 3,375 steps per epoch (batch size 32), or 16,875 steps in total; these step counts follow from the batch sizes and the number of epochs.

$$\eta_t = \eta_{\max} \frac{t}{T_w} \quad (t < T_w), \qquad \eta_t = \eta_{\max} \frac{T - t}{T - T_w} \quad (t \geq T_w) \tag{5.18}$$

5.4.3 Gradient clipping

Before every update the gradients are rescaled if their global Euclidean norm g exceeds a threshold $c = 1.0$ (Equation (5.19)), which prevents occasional very large gradients from destabilising training [13].

$$g \leftarrow g \cdot \min\left(1, \frac{c}{\lVert g \rVert_2}\right) \tag{5.19}$$

5.4.4 Mixed precision and regularisation

The Transformers are trained with automatic mixed precision, in which most arithmetic is performed in 16-bit floating point and a gradient scaler prevents small gradients from underflowing [28]. The recurrent models use 32-bit arithmetic. Regularisation differs between the families: the recurrent models apply dropout with probability 0.5 to the embeddings and to the pooled features, and the Transformers apply a classifier dropout of 0.2; weight decay is shared.

5.4.5 Model selection and evaluation protocol

After every epoch the model is evaluated on the validation split, and the weights of the epoch with the lowest validation loss are stored. After the last epoch these weights are restored and evaluated once on the test split. Because the test split is touched only after the weights are fixed, the test results are an unbiased estimate of performance for the stored model, up to the sampling variation discussed in Section 8.7.

Chapter 6

Experimental Setup

6.1 Environment

The experiments were implemented in Python with PyTorch [34], Hugging Face Transformers [35] and Datasets [29], and scikit-learn [19], together with pandas, NumPy and Matplotlib. Table 6.1 lists the environment recorded in the notebook output. The Python interpreter version and the versions of NumPy, pandas and Matplotlib were not recorded.

Table 6.1: Execution environment recorded in the notebook output

Item	Value
Execution mode	Full dataset (USE_FULL_DATASET = True)
Device	CUDA GPU: Tesla T4, 14.6 GB memory; one GPU used
PyTorch	2.11.0+cu130
Transformers	5.21.0
Datasets	4.8.5
scikit-learn	1.6.1
Random seed	42 (Python, NumPy, PyTorch, CUDA; deterministic settings enabled)
Results folder	./results (JSON files per model and final_comparison.csv)

6.2 Notebook organisation

The experiments are implemented in one notebook that runs from top to bottom. Its sections are: environment setup and configuration, dataset loading, exploratory data analysis and data audit, preprocessing, shared training and evaluation functions, one section for each of the five models, test-set evaluation, confusion matrices, the final model comparison, a results discussion and a conclusion. The executed configuration uses the full official training and test splits. All settings are defined once in the configuration cell.

6.3 Configuration of the models

Table 6.2 compares the settings of the two model families. Within each family all models share the same settings. Between the families the settings differ because a randomly initialised network and a pretrained network require different learning rates and input representations. The comparison therefore does not hold every training condition identical, and it is not a pure architecture comparison.

Table 6.2: Experimental configuration by model family

Setting	Simple RNN / LSTM / GRU	BERT / DistilBERT
Initial weights	Random (including embeddings)	Pretrained encoder; new random classifier
Tokenisation	Word-level regex, lower-case	Pretrained WordPiece (uncased)
Vocabulary	20,000 entries (minimum frequency 3; cap reached; validation OOV 2.87%)	Pretrained vocabulary of the model
Maximum length	64 words	128 tokens

Setting	Simple RNN / LSTM / GRU	BERT / DistilBERT
Embedding / hidden size	128 / 128 per direction (bidirectional)	768 hidden; 12 layers (BERT), 6 layers (DistilBERT)
Pooling	Mean + max over positions	[CLS]-based classification head
Dropout	0.5 (embeddings and before classifier)	Classifier dropout 0.2
Optimiser	AdamW, weight decay 0.01	AdamW, weight decay 0.01
Learning rate	2e-3	2e-5
LR schedule	Linear warm-up (10%) then linear decay	Linear warm-up (10%) then linear decay
Batch size (train / eval)	64 / 128	32 / 128
Epochs	5	5
Gradient clipping	1.0	1.0
Numeric precision	fp32	fp16 mixed precision with gradient scaling
Loss	Cross-entropy	Cross-entropy
Checkpoint selection	Lowest validation loss	Lowest validation loss

6.3.1 Tensor shapes

Table 6.3 lists the tensors that the training functions receive. The recurrent models need the true length of each article so that padded positions can be skipped, and the Transformers need the attention mask for the same purpose.

Table 6.3: Training tensors passed to the models (full dataset)

Model family	Tensors (108,000 training articles)	Shapes
Simple RNN, LSTM, GRU	Word identifiers, true lengths, labels	(108,000 × 64), (108,000), (108,000)
BERT	Token identifiers, attention mask, labels	(108,000 × 128), (108,000 × 128), (108,000)
DistilBERT	Token identifiers, attention mask, labels	(108,000 × 128), (108,000 × 128), (108,000)

6.3.2 Choice of hyperparameters

The hyperparameters in Table 6.2 are fixed values defined in the configuration cell; no search over them was performed. The recurrent models use a larger learning rate (2e-3) because they are trained from random initialisation, whereas the pretrained Transformers are fine-tuned with a small learning rate (2e-5) so that the pretrained weights are changed gradually. The batch size is smaller for the Transformers (32) because of their memory footprint, and the maximum length is longer (128 tokens) because WordPiece produces more tokens than there are words in an article. These choices are common practice; they were not tuned on this task, so a different setting could change the ranking, especially among the three recurrent models.

6.4 Parameter counts

Parameters are counted with one rule for all models: every parameter tensor, including the embedding table and, for the Transformers, the pretrained weights. For the recurrent models the counts can be verified analytically (Table 6.4); the embedding table of 20,000 × 128 = 2,560,000 values accounts for most parameters. All parameters of all models are trainable, but the Transformer weights start from pretraining whereas the recurrent weights are learned from scratch.

Table 6.4: Parameter counts (total = trainable)

Model	Embedding	Recurrent / encoder layers	Classifier / other	Total
Simple RNN	2,560,000	66,048	2,052	2,628,100
LSTM	2,560,000	264,192	2,052	2,826,244
GRU	2,560,000	198,144	2,052	2,760,196
BERT	Included in pretrained weights	Transformer encoder	Classification head	109,485,316
DistilBERT	Included in pretrained weights	Transformer encoder	Classification head	66,956,548

Figure 6.1 plots the same counts on a logarithmic axis. The Transformers have about 24 to 39 times as many parameters as the recurrent models, which is the main driver of the cost differences reported in Chapter 8.

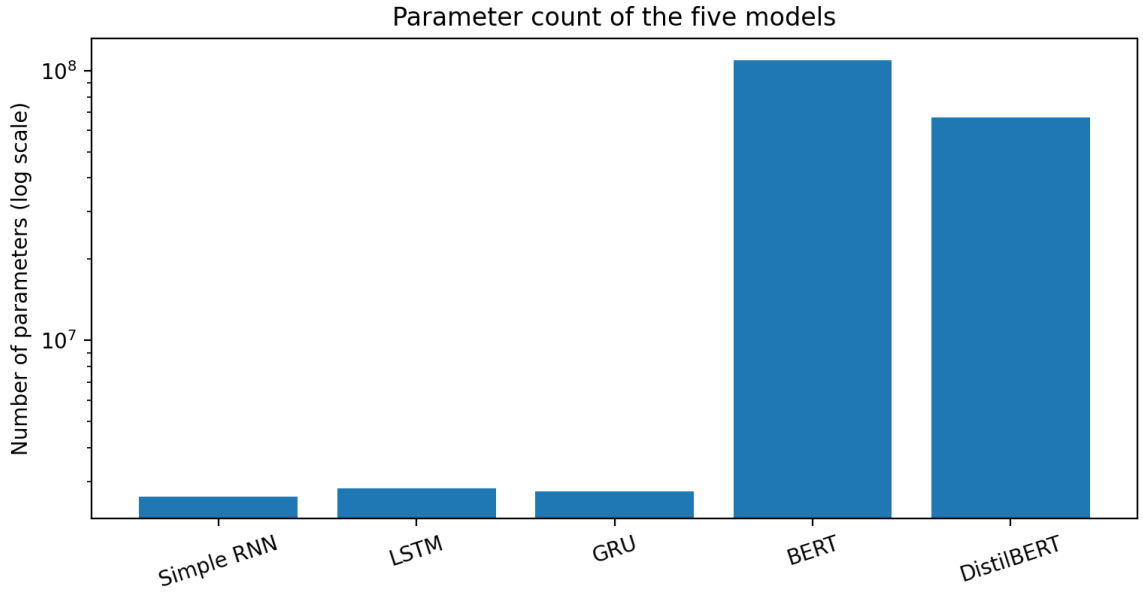


Figure 6.1: Number of parameters of the five models (logarithmic axis; values from the notebook output).

6.5 Measurement of time

Training time is the wall-clock time of the whole epoch loop, including the per-epoch validation passes, measured with time.perf_counter after a CUDA synchronisation, and excluding data preparation and model loading. Inference time is the wall-clock time to predict the entire test set (7,600 articles, batch size 128) after one untimed warm-up batch, again with CUDA synchronisation. Times depend on the hardware, on other processes and on numeric precision, so they are comparable within this run but are not universal constants.

6.6 Reproducibility measures

- **Seeds.** One seed (42) is set for Python, NumPy, PyTorch and CUDA, and it is reset immediately before each model is built.
- **Deterministic settings.** cuDNN is set to deterministic mode with benchmarking disabled.

- **Data order.** The training data loader uses a generator seeded with 42, so every model sees the batches in the same order for a given batch size.
- **Shared partitions.** All models use the same training, validation and test partitions, which are produced once by stratified splits with a fixed seed.
- **Saved outputs.** The notebook writes a training record and a result file for every model, plus a combined comparison table, to a results folder.

These measures make the experiment repeatable in principle. Mixed-precision arithmetic and GPU kernels can still introduce small numerical differences between runs, and a single seed cannot show how much the results vary with the seed.

6.7 Fairness of the comparison

The following design choices make the comparison as fair as the setting allows: all models use the same stratified partitions; the same metric code evaluates them; all are trained for five epochs with AdamW, weight decay, a linear schedule and gradient clipping; the weights of the best validation epoch are restored for every model; the test set is used only once, after training; and the three recurrent models share identical dimensions. The following differences remain and are discussed in Chapter 11: pretraining versus random initialisation, different tokenisers and maximum sequence lengths, different learning rates and batch sizes, fp16 versus fp32, and hardware-dependent timings.

Chapter 7

Evaluation Metrics

7.1 Confusion matrix and the multiclass setting

The confusion matrix of a four-class problem is a 4 × 4 table in which the entry in row i and column j counts the articles of true class i predicted as class j. Metrics for class k are defined in a one-versus-rest manner. The true positives (TP_k) are articles of class k predicted as k; the false positives (FP_k) are articles of other classes predicted as k; the false negatives (FN_k) are articles of class k predicted as another class; and the true negatives (TN_k) are all remaining articles.

7.2 Predictive-performance metrics

Accuracy is the fraction of correctly classified articles (Equation (7.1)), where N is the number of test articles.

$$\text{Accuracy} = \frac{\sum_k TP_k}{N} \tag{7.1}$$

Precision for class k is the fraction of articles predicted as k that truly belong to k, and **recall** is the fraction of articles of class k that are found (Equation (7.2)). The **F1-score** is their harmonic mean (Equation (7.3)).

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \qquad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k} \tag{7.2}$$

$$F1_k = \frac{2\,\text{Precision}_k\,\text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \tag{7.3}$$

Macro-averaged precision, recall and F1-score are the unweighted means of the per-class values; Equation (7.4) shows the macro F1-score. Macro averaging gives every class the same weight regardless of its frequency, so a model cannot hide poor performance on one class behind good performance on the others. On a perfectly balanced test set such as the one used here (1,900 articles per class), macro recall is mathematically equal to accuracy; this equality is visible in the results (Table 8.1). Macro precision and macro F1 can still differ from accuracy.

$$\text{Macro-}F1 = \frac{1}{4}\sum_{k=1}^{4} F1_k \tag{7.4}$$

All metrics are computed with scikit-learn [19] (`accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `confusion_matrix`, `classification_report`) with the class order World, Sports, Business, Sci/Tech and `zero_division = 0`.

7.3 Worked example from a confusion matrix

Table 7.1 applies the definitions to the BERT confusion matrix in Figure 8.3. For the class Business, 1,732 articles are correctly predicted (TP), 120 articles of other classes are

predicted as Business (39 World, 7 Sports and 74 Sci/Tech; FP), and 168 Business articles are predicted as another class (39 + 3 + 126; FN). Precision is therefore 1,732 / (1,732 + 120) = 0.9352 and recall is 1,732 / (1,732 + 168) = 0.9116, which match the classification report printed by the notebook. The values in the table are derived from the matrix and agree with the printed reports for all four classes.

Table 7.1: Per-class counts and metrics for BERT derived from its confusion matrix (7,600 test articles)

Class	TP	FP	FN	TN	Precision	Recall	F1
World	1,808	75	92	5,625	0.9602	0.9516	0.9559
Sports	1,875	18	25	5,682	0.9905	0.9868	0.9887
Business	1,732	120	168	5,580	0.9352	0.9116	0.9232
Sci/Tech	1,794	178	106	5,522	0.9097	0.9442	0.9267

Averaging the four precision values gives the macro precision of 0.9489, and averaging the four F1 values gives the macro F1-score of 0.9486, both as printed by the notebook.

7.4 Averaging strategies and the balanced test set

Multiclass metrics can be averaged in three common ways. Macro averaging takes the unweighted mean over the classes. Weighted averaging weights each class by its number of test articles. Micro averaging pools all decisions before computing the metric, and in single-label multiclass classification micro-averaged precision, recall and F1 all equal accuracy. Because the full test set contains exactly 1,900 articles of each class, the weighted averages equal the macro averages, and macro recall equals accuracy; the notebook's classification reports show identical macro and weighted rows. The metrics therefore differ in what they reveal, not in how they weight the classes, and macro precision and macro F1 are the measures that can differ from accuracy.

7.5 Interpreting the metrics

- **Precision of a class** answers the question: when the model predicts this class, how often is it right? Low precision means that articles of other classes are being absorbed into the class.
- **Recall of a class** answers the question: of the articles that truly belong to this class, how many are found? Low recall means that the class is being missed.
- **F1-score** combines the two with a harmonic mean, so it is high only when both precision and recall are high.
- **Accuracy** is easy to interpret, but it hides which classes are responsible for errors; the per-class reports and the confusion matrix expose them.
- **Differences between models should be read together with the size of the test set: with 7,600 articles, one article changes accuracy by about 0.0132 percentage points (Section 8.7).**

7.6 Computational-efficiency measurements

Three further quantities describe cost rather than predictive quality: training time (seconds), inference time on the test set (seconds, and per article), and the number of parameters. They are reported separately from the predictive metrics and must be interpreted with the caveats of Section 6.5. A derived figure used in the notebook is

accuracy per training second, the test accuracy divided by the training time; it is a descriptive ratio, not a standard metric.

Chapter 8

Results and Analysis

> **Full-dataset results. Every number in this chapter comes from the complete AG News experiment: 108,000 training, 12,000 validation and 7,600 test articles, one seed (42), Tesla T4 GPU. The values are traceable to the notebook's training logs and classification reports.**

8.1 Overall comparison

Table 8.1 reports the test-set predictive metrics, ranked by accuracy, and Table 8.2 the computational costs. BERT achieved the highest accuracy and macro F1-score, DistilBERT was a close second, and the three recurrent models were about 2.5 to 3 percentage points below the Transformers. Among the recurrent models, LSTM was highest, followed very closely by GRU; Simple RNN was lower by about 0.9 percentage points.

Table 8.1: Test-set performance (full dataset, 7,600 test articles), ranked by accuracy

Model	Accuracy	Macro precision	Macro recall	Macro F1
BERT	0.9486	0.9489	0.9486	0.9486
DistilBERT	0.9464	0.9470	0.9464	0.9464
LSTM	0.9212	0.9210	0.9212	0.9209
GRU	0.9211	0.9209	0.9211	0.9208
Simple RNN	0.9121	0.9118	0.9121	0.9118

Table 8.2: Computational cost (full dataset). Per-article inference time is derived from the two-decimal total and is approximate

Model	Training time (s)	Inference time (s)	Inference (ms/article)	Parameters	Best epoch
Simple RNN	125.0	0.32	0.042	2,628,100	4
LSTM	134.0	0.33	0.043	2,826,244	4
GRU	130.2	0.31	0.041	2,760,196	4
BERT	3,404.4	11.09	1.459	109,485,316	2
DistilBERT	1,765.7	5.54	0.729	66,956,548	2

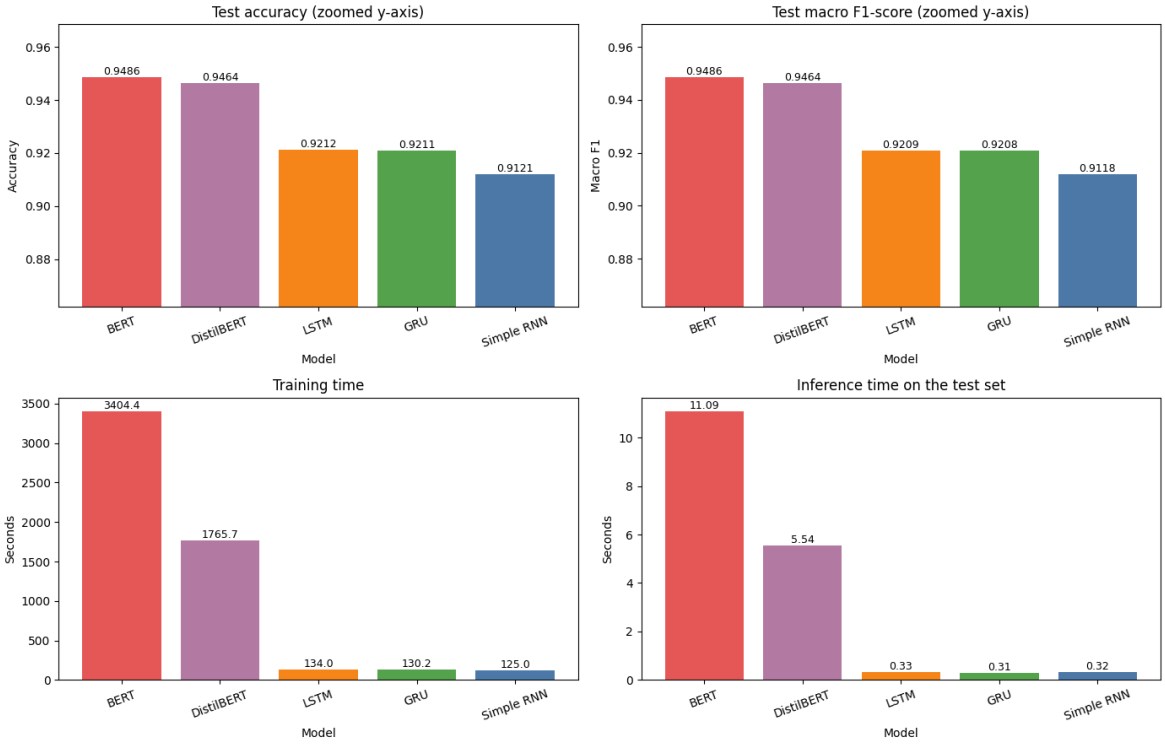


Figure 8.1: Test accuracy, macro F1, training time and inference time for the five models.

Figure 8.1 shows that the Transformers achieve higher accuracy than the recurrent models, at the price of substantially longer training and inference times. BERT used about 25.4 times the training time, 33.2 times the inference time and 38.7 times the parameters of the best recurrent model (LSTM). DistilBERT used about 0.52 times BERT's training time, 0.50 times its inference time and 0.61 times its parameters.

8.2 Per-class results

Table 8.3 lists precision, recall and F1-score for every class and model. Sports is the easiest class for all five models (F1 between 0.9657 and 0.9887), while Business is the hardest class overall (F1 between 0.8783 and 0.9232). The recurrent models show a larger spread across classes than the Transformers.

Table 8.3: Per-class precision, recall and F1-score on the full test set (1,900 articles per class)

Model	Class	Precision	Recall	F1-score
Simple RNN	World	0.9189	0.9063	0.9126
	Sports	0.9529	0.9789	0.9657
	Business	0.8889	0.8679	0.8783
	Sci/Tech	0.8864	0.8953	0.8908
LSTM	World	0.9366	0.9095	0.9228
	Sports	0.9508	0.9863	0.9682
	Business	0.8957	0.8858	0.8907
	Sci/Tech	0.9008	0.9032	0.9020
GRU	World	0.9375	0.9158	0.9265
	Sports	0.9516	0.9837	0.9674
	Business	0.8881	0.8900	0.8891

Model	Class	Precision	Recall	F1-score
	Sci/Tech	0.9062	0.8947	0.9004
BERT	World	0.9602	0.9516	0.9559
	Sports	0.9905	0.9868	0.9887
	Business	0.9352	0.9116	0.9232
	Sci/Tech	0.9097	0.9442	0.9267
DistilBERT	World	0.9558	0.9553	0.9555
	Sports	0.9889	0.9858	0.9873
	Business	0.9424	0.8953	0.9182
	Sci/Tech	0.9011	0.9495	0.9247

8.3 Class-level differences between the model families

Figure 8.2 compares the per-class F1-scores of the five models. The ordering of the classes is similar for all models, with Sports highest and Business lowest. Table 8.4 compares BERT with LSTM, the strongest recurrent model by overall test accuracy, class by class. The Transformer advantage ranges from about 2.05 percentage points for Sports to 3.31 points for World. These are measured differences on one test set and one seed.

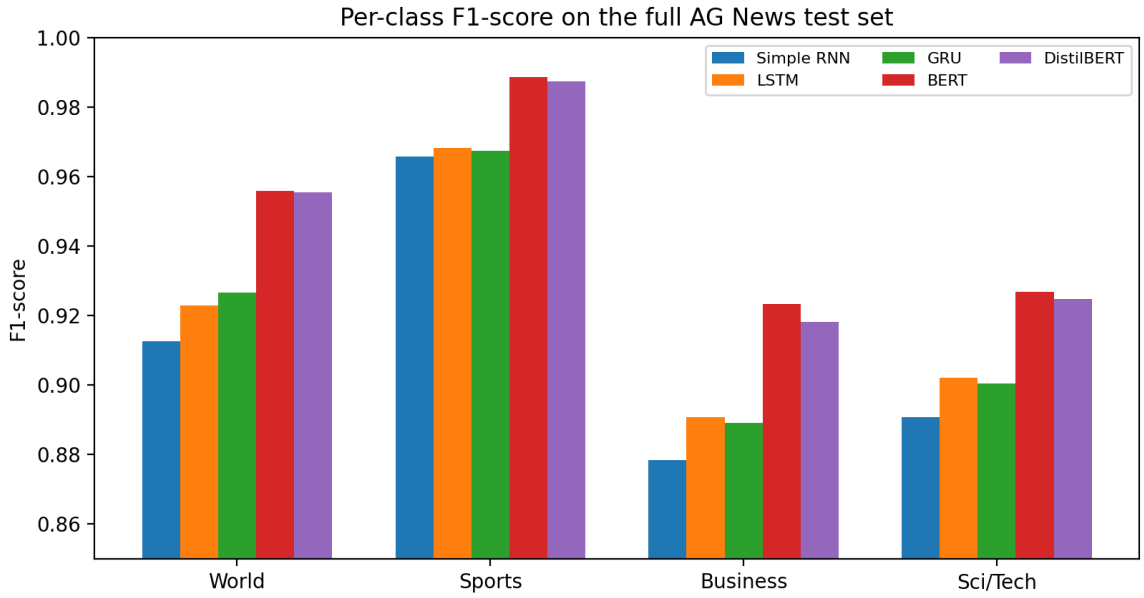


Figure 8.2: Per-class F1-score of the five models on the full test set (values from the classification reports).

Table 8.4: Per-class F1-score of LSTM (best recurrent model) and BERT, and the difference in percentage points

Class	LSTM F1	BERT F1	Difference (points)
World	0.9228	0.9559	+3.31
Sports	0.9682	0.9887	+2.05
Business	0.8907	0.9232	+3.25
Sci/Tech	0.9020	0.9267	+2.47

8.4 Confusion matrices and error patterns

Figure 8.3 shows the confusion matrix of each model; the underlying counts reproduce the printed precision, recall and accuracy values of every model exactly. Table 8.5 lists each model's three most frequent confusions. Confusion between Business and Sci/Tech remains prominent: Business predicted as Sci/Tech is the largest single error for every model (162, 139, 137, 126 and 154 articles for Simple RNN, LSTM, GRU, BERT and DistilBERT respectively). This answers RQ5 for the full test set.

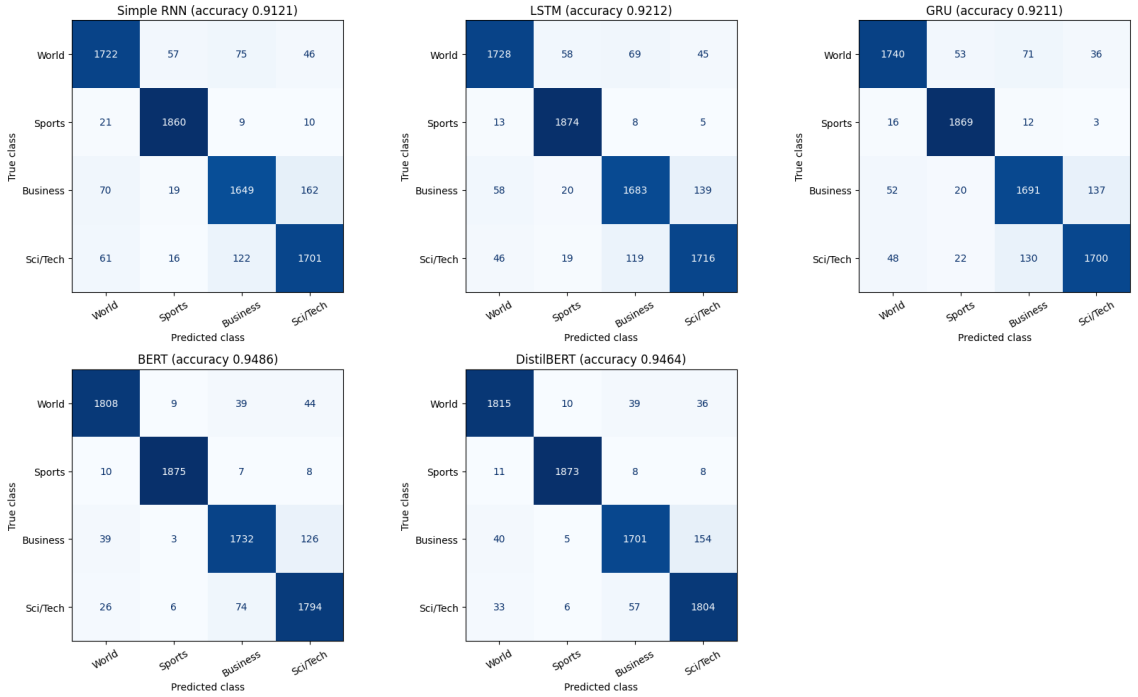


Figure 8.3: Confusion matrices of the five models on the full 7,600-article test set (rows: true class; columns: predicted class).

Table 8.5: Most frequent misclassifications per model (true class → predicted class, article count)

Model	Most frequent	Second	Third
Simple RNN	Business → Sci/Tech (162)	Sci/Tech → Business (122)	World → Business (75)
LSTM	Business → Sci/Tech (139)	Sci/Tech → Business (119)	World → Business (69)
GRU	Business → Sci/Tech (137)	Sci/Tech → Business (130)	World → Business (71)
BERT	Business → Sci/Tech (126)	Sci/Tech → Business (74)	World → Sci/Tech (44)
DistilBERT	Business → Sci/Tech (154)	Sci/Tech → Business (57)	Business → World (40)

8.5 Pairwise differences and efficiency ratios

Table 8.6 lists the difference in test accuracy for every pair of models, in percentage points and in numbers of test articles (one article is about 0.0132 points). It makes explicit which differences are small and which are larger across model families.

Table 8.6: Pairwise differences in test accuracy (first model minus second model)

Comparison	Difference (percentage points)	Difference (test articles)
BERT − DistilBERT	+0.21	+16
BERT − GRU	+2.75	+209
BERT − Simple RNN	+3.64	+277

Comparison	Difference (percentage points)	Difference (test articles)
BERT − LSTM	+2.74	+208
DistilBERT − GRU	+2.54	+193
DistilBERT − Simple RNN	+3.43	+261
DistilBERT − LSTM	+2.53	+192
GRU − Simple RNN	+0.89	+68
GRU − LSTM	-0.01	-1
Simple RNN − LSTM	-0.91	-69

Table 8.7 gives the computational cost of the Transformers relative to the best recurrent model (LSTM) and of DistilBERT relative to BERT. The BERT / LSTM and DistilBERT / BERT ratios are those printed by the notebook from the unrounded measurements; the DistilBERT / LSTM ratios are not printed by the notebook and are computed here from the rounded values in Table 8.2. All ratios are specific to this run.

Table 8.7: Computational cost ratios

Ratio	Training time	Inference time	Parameters
BERT / LSTM	25.4	33.2	38.7
DistilBERT / LSTM	13.2	16.8	23.7
DistilBERT / BERT	0.52	0.50	0.61

Table 8.8 expresses DistilBERT's results as a fraction of BERT's. DistilBERT retained about 99.8% of BERT's accuracy and macro F1 while using 61.2% of its parameters, 51.9% of its training time and 50.0% of its inference time.

Table 8.8: DistilBERT relative to BERT

Measure	BERT	DistilBERT	DistilBERT / BERT
Accuracy	0.9486	0.9464	0.9978
Macro F1-score	0.9486	0.9464	0.9977
Parameters	109,485,316	66,956,548	0.612
Training time (s)	3,404.4	1,765.7	0.519
Inference time (s)	11.09	5.54	0.500

8.6 Ranking and cost–performance trade-off

Table 8.9: Ranking by test accuracy, macro F1 and accuracy per training second (full dataset)

Rank	By accuracy	By macro F1	By accuracy per training second (value)
1	BERT (0.9486)	BERT (0.9486)	Simple RNN (0.00730)
2	DistilBERT (0.9464)	DistilBERT (0.9464)	GRU (0.00708)
3	LSTM (0.9212)	LSTM (0.9209)	LSTM (0.00688)
4	GRU (0.9211)	GRU (0.9208)	DistilBERT (0.00054)
5	Simple RNN (0.9121)	Simple RNN (0.9118)	BERT (0.00028)

The ranking by accuracy and by macro F1 is identical. Ranked by accuracy per training second, the recurrent models rank ahead of the Transformers because they train much faster. The cost–performance picture is therefore a trade-off: BERT is the most accurate, DistilBERT gives up about 0.21 percentage points of accuracy (16 test articles) for roughly half the cost, and the recurrent models are much cheaper to train and run but slightly less accurate.

8.7 Statistical caveat

No statistical significance test was performed, and none is claimed. As a descriptive reference, the binomial standard error of an accuracy estimate on 7,600 articles is about 0.25 percentage points near 0.95 accuracy and about 0.31 points near 0.92 accuracy. Differences of a few tenths of a percentage point should still be interpreted cautiously because only one random seed was used; the test-set size alone does not account for seed-to-seed variation.

8.8 Summary of the results

Table 8.10 summarises, for each research question, what the full-dataset experiments measured and how strongly the evidence supports it.

Table 8.10: Summary of the measured results by research question (full dataset, one seed)

Research question	Measured result	Strength of evidence
RQ1: Transformers versus recurrent models	BERT 0.9486 and DistilBERT 0.9464 versus 0.9212 for LSTM, the strongest recurrent model	Consistent on full test set; one seed
RQ2: effect of gating	LSTM +0.91 points and GRU +0.89 points over Simple RNN	Modest differences; multi-seed confirmation needed
RQ3: cost of the Transformers	BERT used 25.4× training time, 33.2× inference time and 38.7× parameters of LSTM	Measured on one GPU; indicative
RQ4: BERT versus DistilBERT	DistilBERT 0.21 points lower accuracy, with about half the time and 61% of parameters	Small difference; one seed
RQ5: most confused classes	Business → Sci/Tech was the largest single confusion for all models	Observed in all five confusion matrices

Chapter 9

Sample Predictions

9.1 Status of per-article predictions

A chapter of sample predictions with individual articles, their true labels and each model's output would normally follow the results. The submitted notebook, however, does not save or print per-article predictions: it stores only aggregated outputs (predictions are used inside the evaluation function to compute metrics and are then discarded). Because predictions must not be invented, this chapter does not list individual test articles with predicted labels, and the item remains outstanding until the notebook is extended with a prediction-export cell and rerun.

9.2 Aggregate evidence that is available

What the existing evidence does support is an aggregate error analysis, given in Section 8.4. In addition, the dataset examples in Table 4.5 show why Business and Sci/Tech are difficult to separate: an article about a chip-maker's market outlook carries the Sci/Tech label although much of its content concerns industry growth, while Business articles frequently describe companies and deals. This is an interpretation of a labelled example, offered as a hypothesis about the source of the dominant Business–Sci/Tech confusion; it was not tested on the models' actual errors.

9.3 Correctly classified articles per class

Table 9.1 gives the number of test articles of each class that each model classified correctly, read from the diagonal of the confusion matrices (1,900 articles per class). It shows the same pattern as the per-class reports: Sports is classified most reliably and Business least reliably by every model, and the totals equal the accuracies in Table 8.1 multiplied by 7,600.

Table 9.1: Correctly classified test articles per class (out of 1,900 per class; 7,600 in total)

Model	World	Sports	Business	Sci/Tech	Total
Simple RNN	1,722	1,860	1,649	1,701	6,932
LSTM	1,728	1,874	1,683	1,716	7,001
GRU	1,740	1,869	1,691	1,700	7,000
BERT	1,808	1,875	1,732	1,794	7,209
DistilBERT	1,815	1,873	1,701	1,804	7,193

9.4 Distribution of errors across classes

The confusion matrices also show how the misclassified test articles are distributed. Table 9.2 lists, for every model, the number of misclassified articles of each true class and the share of all errors that falls on the Business–Sci/Tech boundary in either direction. All values are read from the confusion matrices in Figure 8.3.

Table 9.2: Misclassified test articles by true class (out of 1,900 per class) and errors on the Business–Sci/Tech boundary

Model	World	Sports	Business	Sci/Tech	Total errors	Business ↔ Sci/Tech
Simple RNN	178	40	251	199	668	284 (42.5%)
LSTM	172	26	217	184	599	258 (43.1%)
GRU	160	31	209	200	600	267 (44.5%)
BERT	92	25	168	106	391	200 (51.2%)
DistilBERT	85	27	199	96	407	211 (51.8%)

Business produced the most errors for every model and Sports the fewest. The Transformers reduced the total number of errors from 599–668 to 391–407, but they reduced the errors between Business and Sci/Tech less than the other errors: these two directions account for 42.5–44.5% of the errors of the recurrent models and for 51.2% (BERT) and 51.8% (DistilBERT) of the errors of the Transformers. The Business–Sci/Tech boundary therefore becomes the dominant remaining source of error once the other confusions are reduced.

Table 9.3: Number of test articles predicted as each class (column totals of the confusion matrices; difference from the true count of 1,900 in brackets)

Model	World	Sports	Business	Sci/Tech
Simple RNN	1,874 (-26)	1,952 (+52)	1,855 (-45)	1,919 (+19)
LSTM	1,845 (-55)	1,971 (+71)	1,879 (-21)	1,905 (+5)
GRU	1,856 (-44)	1,964 (+64)	1,904 (+4)	1,876 (-24)
BERT	1,883 (-17)	1,893 (-7)	1,852 (-48)	1,972 (+72)
DistilBERT	1,899 (-1)	1,894 (-6)	1,805 (-95)	2,002 (+102)

Table 9.3 shows a systematic difference between the model families in which class absorbs the errors. All three recurrent models predicted Sports more often than it occurs (by 52 to 71 articles), whereas both Transformers predicted Sports slightly less often than it occurs and instead over-predicted Sci/Tech (by 72 articles for BERT and 102 for DistilBERT). DistilBERT also under-predicted Business by 95 articles, which matches its lower Business recall (0.8953) and higher Business precision (0.9424) compared with BERT. These are measured tendencies on one test set; why they arise was not investigated.

9.5 Likely sources of error (hypotheses)

Table 9.4 lists characteristics of the data and the pipeline that may contribute to misclassification, together with the evidence that exists for each in the project. Only the rates and examples are measured; whether a characteristic actually causes errors was not tested, because per-article predictions were not saved.

Table 9.4: Possible sources of error and the evidence available in the project

Possible source	Evidence in the project	Status
Overlap between Business and Sci/Tech	Largest off-diagonal confusion for every model (Table 8.5); examples in Table 4.6	Hypothesis
HTML-entity residue in the text	#39; and amp; visible in printed examples; 39 and s among frequent vocabulary entries	Observed in data; effect untested
Words outside the recurrent vocabulary	2.87% of validation words map to <unk>	Rate measured; effect untested
Truncation	1.83% of articles exceed 64 words; 0.74% of sampled articles exceed 128 tokens	Rate measured; effect untested
Short context	Median article length of 37 words	Hypothesis

9.6 Procedure for adding sample predictions

A chapter of individual predictions can be added without changing the experiment, by exporting the test-set predictions that the evaluation function already computes. The procedure would be:

1. Store, for every model, the test-article index, the true label, the predicted label and the predicted probabilities when the test set is evaluated.
2. Export one table per model, or one combined table, to the results folder together with the article text.
3. Select a small number of articles from four groups: correctly classified by all five models, misclassified by all five, misclassified only by the recurrent models, and misclassified only by the Transformers.
4. Compare the selected misclassified articles with the hypotheses in Table 9.4 and report which of them the examples support.
5. Present the selection in this chapter with the true label, each model's prediction and a brief explanation.

Chapter 10

Discussion

This chapter interprets the measured full-dataset evidence against the research questions. Measured findings are stated as such; explanations are marked as hypotheses because the experimental design (Chapter 6) does not isolate their causes.

10.1 Recurrent models versus pretrained Transformers

Measured: both Transformers outperformed all three recurrent models on the aggregate metrics. BERT achieved 0.9486 accuracy, exceeding the strongest recurrent model, LSTM (0.9212), by 2.74 percentage points. DistilBERT achieved 0.9464 accuracy. The difference between model families is measured on the full official test set; the experiment does not isolate which aspects of pretraining, tokenisation, sequence length or optimisation account for the gap.

10.2 Effect of gating

Measured: LSTM (0.9212) and GRU (0.9211) both exceeded Simple RNN (0.9121), by 0.91 and 0.89 percentage points respectively. Their macro F1-scores were 0.9209 and 0.9208 compared with 0.9118 for Simple RNN. The differences are modest; with one seed, the results do not establish that gating will always help.

10.3 Cost of the Transformers

Measured: BERT required 3,404.4 seconds of training and 11.09 seconds for test inference; DistilBERT required 1,765.7 and 5.54 seconds. The best recurrent model, LSTM, required 134.0 seconds of training and 0.33 seconds for inference. BERT therefore used about 25.4 times the training time and 33.2 times the inference time, as well as 38.7 times as many parameters. Timings are hardware- and implementation-dependent.

10.4 BERT versus DistilBERT

Measured: DistilBERT had about 0.21 percentage points lower accuracy than BERT (0.9464 versus 0.9486, based on the exact test counts), while using 61.2% of its parameters, 51.9% of its training time and 50.0% of its inference time. Its macro F1 was about 0.22 points lower. This is a close accuracy-cost trade-off, but one seed is insufficient to establish the reliability of the small difference.

10.5 Class-specific behaviour and misclassification

Sports was the most reliably classified topic, with F1 above 0.96 for every model. Business and Sci/Tech remained the most difficult boundary: Business-to-Sci/Tech was the largest single confusion for all five models. The confusion matrices show the counts, but the project does not save article-level examples for qualitative inspection, so explanations for these errors remain hypotheses.

10.6 Checkpoint selection and reproducibility

Every model is trained for the same five-epoch budget, and the best weights on the validation split are restored before the test set is evaluated (Section 5.4.5). The selected epochs reported by the notebook are: Simple RNN – epoch 4; LSTM – epoch 4; GRU – epoch 4; BERT – epoch 2; DistilBERT – epoch 2. This is a consequence of the minimum-validation-loss rule. The pipeline fixes the random seeds and uses deterministic settings, but GPU operations and mixed-precision training may still not be reproducible bit for bit, and the results come from a single seed.

10.7 Implications of the full-dataset evaluation

The full-dataset experiment uses all 120,000 official training articles to create 108,000 training and 12,000 validation articles, and all 7,600 official test articles for final evaluation. The findings therefore describe the full AG News benchmark partitions rather than a sample drawn from them. They still apply to one seed and one hardware configuration.

10.8 Relationship to published findings

The authors of DistilBERT report that it retains about 97% of BERT's language-understanding performance on the GLUE benchmark while being 40% smaller and 60% faster [11]. On AG News in this project, DistilBERT reached about 99.8% of BERT's accuracy and macro F1, with 61.2% of its parameters, 51.9% of its training time and 50.0% of its inference time (Table 8.8). The quantities are not directly comparable, because the benchmark, metric, hardware and definition of speed differ.

10.9 Choosing a model under different constraints

Table 10.1 translates the measured results into guidance for the constraints a practitioner may face, based on the full-dataset measurements.

Table 10.1: Model choice by constraint, based on the full-dataset measurements

Constraint	Model suggested by the measurements	Measured basis
Highest accuracy is the priority	BERT	Accuracy 0.9486; macro F1 0.9486
Transformer with lower measured cost	DistilBERT	Accuracy 0.9464 with about 52% of BERT's training time and 50% of its inference time
Low inference latency	GRU or LSTM	Approximately 0.31–0.33 seconds for the full test set
Smallest parameter count	Simple RNN	2,628,100 parameters

10.10 Conclusion

This project compared five neural architectures for four-class AG News topic classification: Simple RNN, LSTM and GRU trained from scratch, and BERT and DistilBERT fine-tuned from pretrained weights. All models used the same full-dataset partitions, metrics, five-epoch budget and validation-based checkpoint selection. The experiments used 108,000 training articles, 12,000 validation articles and all 7,600 official test articles with one random seed.

The full-dataset results answer the research questions as follows. BERT (0.9486) and DistilBERT (0.9464) outperformed the strongest recurrent model, LSTM (0.9212). LSTM

and GRU were slightly more accurate than Simple RNN, but the differences are modest and require multi-seed confirmation. BERT required about 25.4 times the training time and 33.2 times the inference time of LSTM, and had 38.7 times its parameters. DistilBERT gave up 0.21 percentage points of accuracy for about half the time and 61.2% of the parameters. Business-to-Sci/Tech was the most frequent single confusion for all models, while Sports was the strongest class.

Practically, BERT is the choice when the highest measured accuracy is the priority, DistilBERT offers a close accuracy-cost alternative, and LSTM or GRU are suitable when low inference latency and a smaller model are more important. These recommendations are based on one seed and one GPU, so repeated runs and broader efficiency tests would strengthen them.

Chapter 11

Limitations

11.1 Limitations identified

The following limitations are supported by the implementation and outputs, unless marked as potential.

- **Single benchmark. AG News contains four English news categories; performance on other domains, languages and label sets has not been evaluated.**
- **Single random seed.** One seed (42) was used; no mean or standard deviation across seeds, no confidence intervals and no significance tests are available.
- **Limited hyperparameter search.** Hyperparameters were fixed in the configuration; no systematic tuning was performed.
- **Uncontrolled differences between families.** Pretraining, tokeniser, maximum sequence length (64 words versus 128 tokens), learning rate (2e-3 versus 2e-5), batch size (64 versus 32) and numeric precision (fp32 versus fp16) differ, so architecture and transfer learning are confounded.
- **Hardware-dependent timings.** Times were measured once on a single T4 GPU; training time includes per-epoch validation; inference was timed with a warm-up batch but without repetitions.
- **Aggregate-only error analysis.** Per-article predictions were not saved, so qualitative error analysis of misclassified articles is not possible with the current outputs.
- **Dataset limitations (potential).** AG News covers a fixed time period and four coarse topics from news sources; the text contains HTML-entity residue; performance on other domains or newer news is untested.
- **Full test set. The official test set contains 7,600 articles (1,900 per class), so one article changes accuracy by about 0.0132 percentage points; small differences may still be affected by seed variation.**

11.2 Threats to validity

- **Internal validity.** The five models differ in more than the architecture (initial weights, tokeniser, maximum length, learning rate, batch size and numeric precision), so a difference in results cannot be attributed to one factor.
- **External validity. The results come from the four-class English AG News benchmark; they may not transfer to other domains, languages or label sets.**
- **Construct validity.** Accuracy and macro F1-score measure agreement with the dataset's labels, which are themselves imperfect at topic boundaries, and the timings measure one implementation on one GPU.
- **Statistical conclusion validity. One seed was used and no significance test was applied; differences of a few tenths of a percentage point should not be treated as conclusive.**

11.3 Effect of the limitations on the conclusions

Table 11.1: Effect of each main limitation on the conclusions and the treatment in this report

Limitation	Effect on the conclusions	Treatment in this report
Single seed	Small metric differences may vary between runs	Small differences are not interpreted as conclusive
Confounded comparison	Pretraining cannot be separated from tokeniser, length and learning rate	Differences stated; explanations marked as hypotheses
Hardware-dependent timings	Cost ratios are indicative	Timings reported with caveats
No per-article predictions	Error causes cannot be verified qualitatively	Error analysis restricted to aggregate counts
One benchmark	Generalisation beyond AG News is unknown	Avoid claims about all text-classification tasks

Chapter 12

Future Enhancements

12.1 Planned extensions

- **Prediction-level error analysis. Export per-article predictions and probabilities to enable qualitative inspection of difficult Business and Sci/Tech examples.**
- **Multiple seeds. Repeat every experiment with several seeds and report mean ± standard deviation; add a paired significance test such as McNemar's test or a bootstrap confidence interval for the accuracy differences.**
- **Hyperparameter optimisation.** Tune learning rate, hidden size, dropout and epochs on the validation set, keeping the test set untouched until the end.
- **Fairer architecture comparison.** Give the recurrent models pretrained word embeddings, or train the recurrent models on the same WordPiece tokens, and measure inference in identical numeric precision.
- **Efficiency measurement.** Report CPU and GPU latency, memory use and FLOPs, with repeated timing runs.
- **Calibration and error analysis.** Save per-article predictions and probabilities, analyse misclassified Business and Sci/Tech articles, and measure calibration (for example expected calibration error).
- **External generalisation.** Evaluate the trained models on news from other sources or periods.

Bibliography

[1] Zhang, X., Zhao, J. and LeCun, Y., 2015. Character-level convolutional networks for text classification. Advances in Neural Information Processing Systems, 28, pp. 649–657.

[2] Mikolov, T., Chen, K., Corrado, G. and Dean, J., 2013. Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.

[3] Pennington, J., Socher, R. and Manning, C. D., 2014. GloVe: global vectors for word representation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, pp. 1532–1543.

[4] Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K. and Zettlemoyer, L., 2018. Deep contextualized word representations. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers), pp. 2227–2237.

[5] Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K., 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pp. 4171–4186.

[6] Howard, J. and Ruder, S., 2018. Universal language model fine-tuning for text classification. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 328–339.

[7] Elman, J. L., 1990. Finding structure in time. Cognitive Science, 14(2), pp. 179–211.

[8] Hochreiter, S. and Schmidhuber, J., 1997. Long short-term memory. Neural Computation, 9(8), pp. 1735–1780.

[9] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y., 2014. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, pp. 1724–1734.

[10] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I., 2017. Attention is all you need. Advances in Neural Information Processing Systems, 30.

[11] Sanh, V., Debut, L., Chaumond, J. and Wolf, T., 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.

[12] Bengio, Y., Simard, P. and Frasconi, P., 1994. Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks, 5(2), pp. 157–166.

[13] Pascanu, R., Mikolov, T. and Bengio, Y., 2013. On the difficulty of training recurrent neural networks. In Proceedings of the 30th International Conference on Machine Learning, PMLR 28(3), pp. 1310–1318.

[14] Schuster, M. and Paliwal, K. K., 1997. Bidirectional recurrent neural networks. IEEE Transactions on Signal Processing, 45(11), pp. 2673–2681.

[15] Gers, F. A., Schmidhuber, J. and Cummins, F., 2000. Learning to forget: continual prediction with LSTM. Neural Computation, 12(10), pp. 2451–2471.

[16] Bahdanau, D., Cho, K. and Bengio, Y., 2015. Neural machine translation by jointly learning to align and translate. In International Conference on Learning Representations. arXiv:1409.0473.

[17] Wu, Y., Schuster, M., Chen, Z., Le, Q. V., Norouzi, M., et al., 2016. Google's neural machine translation system: bridging the gap between human and machine translation. arXiv preprint arXiv:1609.08144.

[18] Hinton, G., Vinyals, O. and Dean, J., 2015. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531.

[19] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., et al., 2011. Scikit-learn: machine learning in Python. Journal of Machine Learning Research, 12, pp. 2825–2830.

[20] Sokolova, M. and Lapalme, G., 2009. A systematic analysis of performance measures for classification tasks. Information Processing & Management, 45(4), pp. 427–437.

[21] Dietterich, T. G., 1998. Approximate statistical tests for comparing supervised classification learning algorithms. Neural Computation, 10(7), pp. 1895–1923.

[22] Dror, R., Baumer, G., Shlomov, S. and Reichart, R., 2018. The hitchhiker's guide to testing statistical significance in natural language processing. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1383–1392.

[23] Kingma, D. P. and Ba, J., 2015. Adam: a method for stochastic optimization. In International Conference on Learning Representations. arXiv:1412.6980.

[24] Loshchilov, I. and Hutter, F., 2019. Decoupled weight decay regularization. In International Conference on Learning Representations. arXiv:1711.05101.

[25] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R., 2014. Dropout: a simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15, pp. 1929–1958.

[26] He, K., Zhang, X., Ren, S. and Sun, J., 2016. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778.

[27] Ba, J. L., Kiros, J. R. and Hinton, G. E., 2016. Layer normalization. arXiv preprint arXiv:1607.06450.

[28] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., et al., 2018. Mixed precision training. In International Conference on Learning Representations. arXiv:1710.03740.

[29] Lhoest, Q., Villanova del Moral, A., Jernite, Y., Thakur, A., von Platen, P., et al., 2021. Datasets: a community library for natural language processing. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 175–184.

[30] Hugging Face, 2024. fancyzhx/ag_news [dataset]. https://huggingface.co/datasets/fancyzhx/ag_news (dataset version accessed through the datasets library in the project notebook).

[31] Rumelhart, D. E., Hinton, G. E. and Williams, R. J., 1986. Learning representations by back-propagating errors. Nature, 323, pp. 533–536.

[32] Werbos, P. J., 1990. Backpropagation through time: what it does and how to do it. Proceedings of the IEEE, 78(10), pp. 1550–1560.

[33] Goodfellow, I., Bengio, Y. and Courville, A., 2016. Deep Learning. Cambridge, MA: MIT Press.

[34] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., et al., 2019. PyTorch: an imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32.

[35] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., et al., 2020. Transformers: state-of-the-art natural language processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 38–45.

Appendix A

Source Code

This appendix reproduces all 23 code cells of the executed full-dataset notebook AG_News_RNN_vs_Transformers_final_wid_full_dataset.ipynb in their original order and without modification. The executed configuration has USE_FULL_DATASET = True, so the branch that draws a small debug subset is present in the code but was not executed; every result in this report comes from the full official training and test splits. Only the source code is listed; cell outputs are not repeated here, because the printed results are reported in Chapters 4 to 9 and the four figures produced by the notebook appear as Figures 4.1, 4.2, 8.1 and 8.3. The explanatory markdown cells of the notebook are not repeated.

A.1 Environment setup and library imports

Code (notebook code cell 1)

```
import os, sys, time, json, random, re, gc, copy, warnings, importlib,
    subprocess, textwrap
from collections import Counter
warnings.filterwarnings("ignore")

def ensure(pip_name, import_name=None):
    # Install a package only if it cannot be imported.
    try:
        importlib.import_module(import_name or pip_name)
    except ImportError:
        subprocess.check_call([sys.executable, "-m", "pip", "install", "-q",
    pip_name])

ensure("datasets")
ensure("transformers")
ensure("scikit-learn", "sklearn")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display, Markdown

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

import datasets as hf_datasets
import transformers
from datasets import load_dataset
from transformers import (AutoTokenizer, BertForSequenceClassification,
                          DistilBertForSequenceClassification,
                          get_linear_schedule_with_warmup)
from transformers.utils import logging as hf_logging
hf_logging.set_verbosity_error()      # hide the harmless "classifier weights
    newly initialized" warning

import sklearn
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
    f1_score,
                             confusion_matrix, classification_report,
    ConfusionMatrixDisplay)

print("Libraries imported.")
print(f"torch {torch.__version__} | transformers {transformers.__version__} |
    datasets {hf_datasets.__version__} | sklearn {sklearn.__version__}")
```

A.2 Experiment configuration and dataset mode

Code (notebook code cell 2)

```
# ======================= MAIN SWITCH =======================
USE_FULL_DATASET = True     # False = small debug subset, True = full AG News
    (final results)
# ============================================================

SEED = 42
VAL_FRACTION = 0.10          # share of the official training set kept for
    validation
DEBUG_TRAIN_SIZE = 8000      # used only when USE_FULL_DATASET = False
DEBUG_TEST_SIZE = 2000

# --- Recurrent models: identical settings for RNN, LSTM and GRU (fair
    comparison) ---
MAX_VOCAB_SIZE = 20000
MIN_WORD_FREQ = 3
RNN_MAX_LEN = 64             # words per article (longer articles are
    truncated)
EMBED_DIM = 128
HIDDEN_DIM = 128             # per direction (the recurrent layer is
    bidirectional)
DROPOUT = 0.5                # dropout on the embeddings and before the
    classifier (reduces overfitting)
RNN_LR = 2e-3
RNN_WEIGHT_DECAY = 0.01
RNN_BATCH_SIZE = 64

# --- Transformer models: identical settings for BERT and DistilBERT ---
TR_MAX_LEN = 128             # sub-word tokens per article
TR_LR = 2e-5
TR_BATCH_SIZE = 32
TR_WEIGHT_DECAY = 0.01
TR_CLASSIFIER_DROPOUT = 0.2  # extra dropout before the new classification
    layer (reduces overfitting)

EVAL_BATCH_SIZE = 128        # same for validation/test of ALL models

EPOCHS = 5                   # SAME number of epochs for ALL five models (fair
    comparison)
RNN_EPOCHS = TR_EPOCHS = EPOCHS

RESULTS_DIR = "/kaggle/working/results" if os.path.exists("/kaggle/working")
    else "./results"
os.makedirs(RESULTS_DIR, exist_ok=True)

mode = "FULL DATASET" if USE_FULL_DATASET else f"DEBUG SUBSET
    ({DEBUG_TRAIN_SIZE} train / {DEBUG_TEST_SIZE} test)"
```

```
print("Run mode      :", mode)
print("Epochs (all 5 models):", EPOCHS)
print("Results folder:", RESULTS_DIR)
if not USE_FULL_DATASET:
    print("\nNOTE: debug results are NOT final. Set USE_FULL_DATASET = True
    and re-run for your report.")
```

A.3 Reproducibility setup and device configuration

Code (notebook code cell 3)

```
def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed(SEED)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", DEVICE)
if DEVICE.type == "cuda":
    props = torch.cuda.get_device_properties(0)
    print("GPU name        :", props.name)
    print("GPU memory (GB) :", round(props.total_memory / 1024**3, 1))
    print("GPUs visible    :", torch.cuda.device_count(), "(this notebook uses
    only GPU 0)")
else:
    print("WARNING: no GPU found. BERT/DistilBERT will be very slow on CPU.
    Enable a GPU in the Kaggle settings.")
```

A.4 AG News dataset loading

Code (notebook code cell 4)

```
raw = load_dataset("fancyzhx/ag_news")
print(raw)
print("\nFeatures:", raw["train"].features)

CLASS_NAMES = raw["train"].features["label"].names
EXPECTED = ["World", "Sports", "Business", "Sci/Tech"]
print("\nLabel names from the dataset:", CLASS_NAMES)
assert CLASS_NAMES == EXPECTED, "Unexpected label order!"
NUM_CLASSES = len(CLASS_NAMES)
id2label = {i: n for i, n in enumerate(CLASS_NAMES)}
label2id = {n: i for i, n in enumerate(CLASS_NAMES)}
print("id2label:", id2label)
```

A.5 Dataset conversion and label mapping

Code (notebook code cell 5)

```
train_full_df = raw["train"].to_pandas()
test_full_df = raw["test"].to_pandas()

print("Train shape:", train_full_df.shape, "| Test shape:",
    test_full_df.shape)
print("Columns    :", list(train_full_df.columns))
print("\nData types:\n", train_full_df.dtypes)
```

```
for split_name, df_ in [("TRAIN", train_full_df), ("TEST", test_full_df)]:
    counts = df_["label"].value_counts().sort_index()
    table = pd.DataFrame({"label_id": counts.index,
                          "class": [id2label[i] for i in counts.index],
                          "count": counts.values,
                          "percent": (counts.values / len(df_) *
    100).round(2)})
    print(f"\nClass distribution ({split_name}):")
    display(table)

print("\nFirst 3 training records:")
display(train_full_df.head(3))

print("\nLabel mapping check (one example per label id):")
for lid in range(NUM_CLASSES):
    example = train_full_df[train_full_df["label"] == lid]["text"].iloc[0]
    print(f"  label {lid} -> {id2label[lid]:9s} | {example[:110]}...")
```

A.6 Class distribution visualisation

Code (notebook code cell 6)

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, (title, df_) in zip(axes, [("Train (official)", train_full_df), ("Test
    (official)", test_full_df)]):
    counts = df_["label"].value_counts().sort_index()
    ax.bar([id2label[i] for i in counts.index], counts.values,
    color=["#4C78A8", "#F58518", "#54A24B", "#E45756"])
    ax.set_title(f"Class distribution: {title}")
    ax.set_xlabel("Class")
    ax.set_ylabel("Number of articles")
    for i, v in enumerate(counts.values):
        ax.text(i, v, str(v), ha="center", va="bottom")
plt.tight_layout()
plt.show()
```

A.7 Article length and token-length analysis

Code (notebook code cell 7)

```
bert_tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

train_full_df["n_words"] = train_full_df["text"].str.split().str.len()
sample_df = train_full_df.sample(n=min(20000, len(train_full_df)),
    random_state=SEED)
sample_tokens = bert_tokenizer(sample_df["text"].tolist(),
    add_special_tokens=True, truncation=False)["input_ids"]
sample_df = sample_df.assign(n_tokens=[len(x) for x in sample_tokens])

print("Words per article:\n", train_full_df["n_words"].describe().round(1))
print("\nBERT tokens per article (20k sample):\n",
    sample_df["n_tokens"].describe().round(1))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(train_full_df["n_words"], bins=60, color="#4C78A8")
axes[0].axvline(RNN_MAX_LEN, color="red", linestyle="--", label=f"RNN_MAX_LEN
    = {RNN_MAX_LEN}")
axes[0].set_title("Article length in words")
axes[0].set_xlabel("Words"); axes[0].set_ylabel("Number of articles");
    axes[0].legend()
axes[1].hist(sample_df["n_tokens"], bins=60, color="#F58518")
```

```
axes[1].axvline(TR_MAX_LEN, color="red", linestyle="--", label=f"TR_MAX_LEN =
    {TR_MAX_LEN}")
axes[1].set_title("Article length in BERT tokens (20k sample)")
axes[1].set_xlabel("Tokens"); axes[1].set_ylabel("Number of articles");
    axes[1].legend()
plt.tight_layout()
plt.show()

print(f"Articles longer than {RNN_MAX_LEN} words (truncated for RNN/LSTM/GRU):
    {(train_full_df['n_words'] > RNN_MAX_LEN).mean()*100:.2f}%")
print(f"Articles longer than {TR_MAX_LEN} tokens (truncated for
    BERT/DistilBERT): {(sample_df['n_tokens'] > TR_MAX_LEN).mean()*100:.2f}%")

print("\nMean words per class:")
display(train_full_df.groupby(train_full_df["label"].map(id2label))["n_words"]
    .mean().round(1).rename("mean_words").to_frame())
```

A.8 Sample text inspection by class

Code (notebook code cell 8)

```
for lid in range(NUM_CLASSES):
    print(f"=== {id2label[lid]} ===")
    examples = train_full_df[train_full_df["label"] == lid].sample(2,
    random_state=SEED)["text"]
    for ex in examples:
        print(textwrap.fill("- " + ex, width=110, subsequent_indent="  "))
    print()
```

A.9 Dataset quality and duplicate-text audit

Code (notebook code cell 9)

```
for name_, df_ in [("train", train_full_df), ("test", test_full_df)]:
    print(f"[{name_}] missing text: {df_['text'].isna().sum()} | missing
    label: {df_['label'].isna().sum()} | "
          f"empty text: {(df_['text'].fillna('').str.strip() == '').sum()}")

dup_train = train_full_df.duplicated(subset="text").sum()
dup_test = test_full_df.duplicated(subset="text").sum()
print(f"\nExact duplicate texts inside train: {dup_train} | inside test:
    {dup_test}")

label_per_text = train_full_df.groupby("text")["label"].nunique()
print("Texts that appear with more than one label in train:",
    int((label_per_text > 1).sum()))

overlap = set(train_full_df["text"]).intersection(set(test_full_df["text"]))
print("Texts present in BOTH train and test:", len(overlap))
```

A.10 Text cleaning and stratified data splitting

Code (notebook code cell 10)

```
def clean_df(df_):
    df_ = df_.dropna(subset=["text", "label"]).copy()
    df_["text"] = df_["text"].astype(str).str.strip()
    df_ = df_[df_["text"] != ""]
    df_["label"] = df_["label"].astype(int)
    return df_.reset_index(drop=True)

train_pool = clean_df(train_full_df[["text", "label"]])
```

```
test_df = clean_df(test_full_df[["text", "label"]])
print(f"After cleaning -> train pool: {len(train_pool)} | test:
    {len(test_df)}")

if not USE_FULL_DATASET:
    train_pool, _ = train_test_split(train_pool, train_size=DEBUG_TRAIN_SIZE,
    stratify=train_pool["label"], random_state=SEED)
    test_df, _ = train_test_split(test_df, train_size=DEBUG_TEST_SIZE,
    stratify=test_df["label"], random_state=SEED)
    train_pool = train_pool.reset_index(drop=True)
    test_df = test_df.reset_index(drop=True)

train_df, val_df = train_test_split(train_pool, test_size=VAL_FRACTION,
    stratify=train_pool["label"], random_state=SEED)
train_df = train_df.reset_index(drop=True)
val_df = val_df.reset_index(drop=True)

split_summary = pd.DataFrame({
    "split": ["train", "validation", "test (untouched until Section 11)"],
    "size": [len(train_df), len(val_df), len(test_df)],
})
for lid in range(NUM_CLASSES):
    split_summary[id2label[lid]] = [(d["label"] == lid).sum() for d in
    (train_df, val_df, test_df)]
display(split_summary)

train_texts, train_labels = train_df["text"].tolist(),
    train_df["label"].tolist()
val_texts, val_labels = val_df["text"].tolist(), val_df["label"].tolist()
test_texts, test_labels = test_df["text"].tolist(), test_df["label"].tolist()
```

A.11 Vocabulary construction and recurrent-model encoding

Code (notebook code cell 11)

```
TOKEN_PATTERN = re.compile(r"[a-z0-9]+(?:'[a-z]+)?")
def simple_tokenize(text):
    return TOKEN_PATTERN.findall(text.lower())

counter = Counter()
for t in train_texts:
    counter.update(simple_tokenize(t))

PAD_ID, UNK_ID = 0, 1
itos = ["<pad>", "<unk>"] + [w for w, c in counter.most_common() if c >=
    MIN_WORD_FREQ][: MAX_VOCAB_SIZE - 2]
stoi = {w: i for i, w in enumerate(itos)}
VOCAB_SIZE = len(itos)
print("Vocabulary size:", VOCAB_SIZE, "| most common words:", itos[2:12])

def encode_for_rnn(texts, max_len=RNN_MAX_LEN):
    ids = np.zeros((len(texts), max_len), dtype=np.int64)        # 0 = <pad>
    lengths = np.zeros(len(texts), dtype=np.int64)
    for i, t in enumerate(texts):
        toks = [stoi.get(w, UNK_ID) for w in simple_tokenize(t)][:max_len]   #
    truncate
        if len(toks) == 0:
            toks = [UNK_ID]
        ids[i, :len(toks)] = toks
    # pad on the right
```

```
        lengths[i] = len(toks)
    return torch.from_numpy(ids), torch.from_numpy(lengths)

example = train_texts[0]
ex_ids, ex_len = encode_for_rnn([example])
print("\nExample text   :", example[:120], "...")
print("Word tokens    :", simple_tokenize(example)[:12], "...")
print("Encoded ids    :", ex_ids[0][:12].tolist(), "... | true length:",
    ex_len.item())

val_tokens = [w for t in val_texts for w in simple_tokenize(t)]
oov_rate = np.mean([w not in stoi for w in val_tokens])
print(f"\nOut-of-vocabulary rate on validation words: {oov_rate*100:.2f}%")

DATA = {"rnn": {}}
for split, texts, labels in [("train", train_texts, train_labels), ("val",
    val_texts, val_labels), ("test", test_texts, test_labels)]:
    ids, lengths = encode_for_rnn(texts)
    DATA["rnn"][split] = (ids, lengths, torch.tensor(labels,
    dtype=torch.long))
print("RNN tensor shapes (train):", [tuple(x.shape) for x in
    DATA["rnn"]["train"]])
```

A.12 Transformer tokenisation and input encoding

Code (notebook code cell 12)

```
distil_tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

def encode_for_transformer(tokenizer, texts, labels, max_len=TR_MAX_LEN):
    enc = tokenizer(texts, truncation=True, max_length=max_len,
    padding="max_length", return_tensors="pt")
    return (enc["input_ids"], enc["attention_mask"], torch.tensor(labels,
    dtype=torch.long))

DATA["bert"], DATA["distilbert"] = {}, {}
for split, texts, labels in [("train", train_texts, train_labels), ("val",
    val_texts, val_labels), ("test", test_texts, test_labels)]:
    DATA["bert"][split] = encode_for_transformer(bert_tokenizer, texts,
    labels)
    DATA["distilbert"][split] = encode_for_transformer(distil_tokenizer,
    texts, labels)

print("BERT train tensors      :", [tuple(x.shape) for x in
    DATA["bert"]["train"]])
print("DistilBERT train tensors:", [tuple(x.shape) for x in
    DATA["distilbert"]["train"]])
sample_ids = DATA["bert"]["train"][0][0]
print("\nExample BERT tokens:",
    bert_tokenizer.convert_ids_to_tokens(sample_ids[:20].tolist()), "...")
```

A.13 Data loaders and shared training/evaluation utilities

Code (notebook code cell 13)

```
def make_loader(tensors, batch_size, shuffle):
    generator = torch.Generator()
    generator.manual_seed(SEED)
    return DataLoader(TensorDataset(*tensors), batch_size=batch_size,
    shuffle=shuffle,
                      generator=generator if shuffle else None)
```

```
def sync():
    if DEVICE.type == "cuda":
        torch.cuda.synchronize()

def forward_batch(model, batch, kind):
    # kind = "rnn" -> (ids, lengths, labels);  kind = "transformer" -> (ids,
    attention_mask, labels)
    if kind == "rnn":
        x, lengths, y = batch
        logits = model(x.to(DEVICE), lengths)
    else:
        ids, mask, y = batch
        logits = model(input_ids=ids.to(DEVICE),
    attention_mask=mask.to(DEVICE)).logits
    return logits, y.to(DEVICE)

@torch.no_grad()
def predict(model, loader, kind, use_amp=False):
    model.eval()
    criterion = nn.CrossEntropyLoss(reduction="sum")
    total_loss, all_preds, all_labels = 0.0, [], []
    sync()
    start = time.perf_counter()
    for batch in loader:
        with torch.autocast(device_type=DEVICE.type, enabled=use_amp):
            logits, y = forward_batch(model, batch, kind)
        total_loss += criterion(logits.float(), y).item()
        all_preds.append(logits.argmax(dim=1).cpu())
        all_labels.append(y.cpu())
    sync()
    elapsed = time.perf_counter() - start
    preds = torch.cat(all_preds).numpy()
    labels = torch.cat(all_labels).numpy()
    return total_loss / len(labels), preds, labels, elapsed

def count_parameters(model):
    # Convention used for ALL models: every parameter tensor in the model,
    including
    # the embedding table and (for BERT/DistilBERT) the pretrained weights.
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    return total, trainable

def train_model(name, kind, build_fn, data, epochs, lr, batch_size,
    use_amp=False, weight_decay=0.0, use_scheduler=False):
    set_seed(SEED)
    model = build_fn().to(DEVICE)
    total_params, trainable_params = count_parameters(model)
    print(f"[{name}] parameters: total={total_params:,} |
    trainable={trainable_params:,} | batch size={batch_size}")

    train_loader = make_loader(data["train"], batch_size, shuffle=True)
    val_loader = make_loader(data["val"], EVAL_BATCH_SIZE, shuffle=False)
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr,
    weight_decay=weight_decay)
    criterion = nn.CrossEntropyLoss()
    scheduler = None
    if use_scheduler:
```

```
        total_steps = len(train_loader) * epochs
        scheduler = get_linear_schedule_with_warmup(optimizer, int(0.1 *
    total_steps), total_steps)
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)

    history, best_val_loss, best_epoch, best_state = [], float("inf"), 0, None
    sync()
    train_start = time.perf_counter()
    for epoch in range(1, epochs + 1):
        model.train()
        running_loss, n_seen = 0.0, 0
        for batch in train_loader:
            optimizer.zero_grad(set_to_none=True)
            with torch.autocast(device_type=DEVICE.type, enabled=use_amp):
                logits, y = forward_batch(model, batch, kind)
            loss = criterion(logits.float(), y)
            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            scaler.step(optimizer)
            scaler.update()
            if scheduler is not None:
                scheduler.step()
            running_loss += loss.item() * y.size(0)
            n_seen += y.size(0)
        train_loss = running_loss / n_seen
        val_loss, val_preds, val_true, _ = predict(model, val_loader, kind,
    use_amp)
        val_acc = accuracy_score(val_true, val_preds)
        history.append({"epoch": epoch, "train_loss": train_loss, "val_loss":
    val_loss, "val_accuracy": val_acc})
        print(f"[{name}] epoch {epoch}/{epochs} | train loss {train_loss:.4f}
    | val loss {val_loss:.4f} | val acc {val_acc:.4f}")
        if val_loss < best_val_loss:
            best_val_loss, best_epoch = val_loss, epoch
            best_state = {k: v.detach().cpu().clone() for k, v in
    model.state_dict().items()}
    sync()
    train_time = time.perf_counter() - train_start
    print(f"[{name}] training time: {train_time:.1f} s | best epoch (by lowest
    validation loss): {best_epoch}")

    del model, optimizer, scaler
    gc.collect()
    if DEVICE.type == "cuda":
        torch.cuda.empty_cache()
    return {"history": history, "best_epoch": best_epoch, "best_state":
    best_state, "train_time_sec": train_time,
            "params_total": total_params, "params_trainable":
    trainable_params, "batch_size_used": batch_size}

def run_training(name, kind, build_fn, data, epochs, lr, batch_size,
    **kwargs):
    bs = batch_size
    for attempt in range(1, 4):
        out_of_memory = False
        try:
            return train_model(name, kind, build_fn, data, epochs, lr, bs,
    **kwargs)
        except torch.cuda.OutOfMemoryError:
```

```
            out_of_memory = True
        if out_of_memory:
            gc.collect(); torch.cuda.empty_cache()
            bs = max(1, bs // 2)
            print(f"[{name}] GPU out of memory (attempt {attempt}). Retrying
    with batch size {bs} ...")
    raise RuntimeError(f"[{name}] still out of memory. Reduce TR_MAX_LEN or
    TR_BATCH_SIZE in the configuration cell.")

TRAINED = {}      # filled in Sections 6-10: best weights + training history
    for every model
MODEL_NAMES = ["Simple RNN", "LSTM", "GRU", "BERT", "DistilBERT"]
def slug(name): return name.lower().replace(" ", "_")

def save_training_record(name, rec):
    with open(os.path.join(RESULTS_DIR, f"{slug(name)}_training.json"), "w")
    as f:
        keep = ["history", "best_epoch", "train_time_sec", "params_total",
    "params_trainable", "batch_size_used"]
        json.dump({k: rec[k] for k in keep}, f, indent=2)
print("Shared functions ready.")
```

A.14 Recurrent neural network classifier and Simple RNN training

Code (notebook code cell 14)

```
class RecurrentClassifier(nn.Module):
    def __init__(self, rnn_type, vocab_size, embed_dim, hidden_dim,
    num_classes, dropout, pad_idx=0):
        super().__init__()
        self.rnn_type = rnn_type
        self.embedding = nn.Embedding(vocab_size, embed_dim,
    padding_idx=pad_idx)
        rnn_class = {"rnn": nn.RNN, "lstm": nn.LSTM, "gru": nn.GRU}[rnn_type]
        self.rnn = rnn_class(embed_dim, hidden_dim, batch_first=True,
    bidirectional=True)
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim * 2 * 2, num_classes)      # (2
    directions) x (mean + max pooling)

    def forward(self, x, lengths):
        emb = self.dropout(self.embedding(x))
    # (batch, len, embed_dim)
        packed = nn.utils.rnn.pack_padded_sequence(emb, lengths.cpu(),
    batch_first=True, enforce_sorted=False)
        out, _ = self.rnn(packed)
        out, _ = nn.utils.rnn.pad_packed_sequence(out, batch_first=True,
    total_length=x.size(1))  # (batch, len, 2*hidden)
        lens = lengths.to(x.device)
        mask = (torch.arange(x.size(1), device=x.device)[None, :] < lens[:,
    None]).unsqueeze(-1)  # True for real words
        mean_pool = (out * mask).sum(dim=1) / lens.unsqueeze(1).clamp(min=1)
        max_pool = out.masked_fill(~mask, -1e4).max(dim=1).values
        features = torch.cat([mean_pool, max_pool], dim=1)
        return self.fc(self.dropout(features))
    # (batch, num_classes)

def build_rnn():
    return RecurrentClassifier("rnn", VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM,
    NUM_CLASSES, DROPOUT)
```

```
print(build_rnn())

rec = run_training("Simple RNN", "rnn", build_rnn, DATA["rnn"], RNN_EPOCHS,
    RNN_LR, RNN_BATCH_SIZE,
                       weight_decay=RNN_WEIGHT_DECAY, use_scheduler=True)
TRAINED["Simple RNN"] = {**rec, "kind": "rnn", "build_fn": build_rnn,
    "data_key": "rnn", "use_amp": False}
save_training_record("Simple RNN", TRAINED["Simple RNN"])
display(pd.DataFrame(rec["history"]).round(4))
```

A.15 LSTM model construction and training

Code (notebook code cell 15)

```
def build_lstm():
    return RecurrentClassifier("lstm", VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM,
    NUM_CLASSES, DROPOUT)

print(build_lstm())
rec = run_training("LSTM", "rnn", build_lstm, DATA["rnn"], RNN_EPOCHS, RNN_LR,
    RNN_BATCH_SIZE,
                       weight_decay=RNN_WEIGHT_DECAY, use_scheduler=True)
TRAINED["LSTM"] = {**rec, "kind": "rnn", "build_fn": build_lstm, "data_key":
    "rnn", "use_amp": False}
save_training_record("LSTM", TRAINED["LSTM"])
display(pd.DataFrame(rec["history"]).round(4))
```

A.16 GRU model construction and training

Code (notebook code cell 16)

```
def build_gru():
    return RecurrentClassifier("gru", VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM,
    NUM_CLASSES, DROPOUT)

print(build_gru())
rec = run_training("GRU", "rnn", build_gru, DATA["rnn"], RNN_EPOCHS, RNN_LR,
    RNN_BATCH_SIZE,
                       weight_decay=RNN_WEIGHT_DECAY, use_scheduler=True)
TRAINED["GRU"] = {**rec, "kind": "rnn", "build_fn": build_gru, "data_key":
    "rnn", "use_amp": False}
save_training_record("GRU", TRAINED["GRU"])
display(pd.DataFrame(rec["history"]).round(4))
```

A.17 BERT model construction and fine-tuning

Code (notebook code cell 17)

```
def build_bert():
    return BertForSequenceClassification.from_pretrained("bert-base-uncased",
    num_labels=NUM_CLASSES,
                                                         id2label=id2label,
    label2id=label2id,

    classifier_dropout=TR_CLASSIFIER_DROPOUT)

use_amp_tr = DEVICE.type == "cuda"
rec = run_training("BERT", "transformer", build_bert, DATA["bert"], TR_EPOCHS,
    TR_LR, TR_BATCH_SIZE,
                   use_amp=use_amp_tr, weight_decay=TR_WEIGHT_DECAY,
    use_scheduler=True)
```

```
TRAINED["BERT"] = {**rec, "kind": "transformer", "build_fn": build_bert,
    "data_key": "bert", "use_amp": use_amp_tr}
save_training_record("BERT", TRAINED["BERT"])
display(pd.DataFrame(rec["history"]).round(4))
```

A.18 DistilBERT model construction and fine-tuning

Code (notebook code cell 18)

```
def build_distilbert():
    return DistilBertForSequenceClassification.from_pretrained("distilbert-
    base-uncased", num_labels=NUM_CLASSES,

    id2label=id2label, label2id=label2id,

    seq_classif_dropout=TR_CLASSIFIER_DROPOUT)

rec = run_training("DistilBERT", "transformer", build_distilbert,
    DATA["distilbert"], TR_EPOCHS, TR_LR, TR_BATCH_SIZE,
                   use_amp=use_amp_tr, weight_decay=TR_WEIGHT_DECAY,
    use_scheduler=True)
TRAINED["DistilBERT"] = {**rec, "kind": "transformer", "build_fn":
    build_distilbert, "data_key": "distilbert", "use_amp": use_amp_tr}
save_training_record("DistilBERT", TRAINED["DistilBERT"])
display(pd.DataFrame(rec["history"]).round(4))
```

A.19 Test-set evaluation and metric calculation

Code (notebook code cell 19)

```
def evaluate_on_test(name):
    info = TRAINED[name]
    model = info["build_fn"]().to(DEVICE)
    model.load_state_dict(info["best_state"])
    loader = make_loader(DATA[info["data_key"]]["test"], EVAL_BATCH_SIZE,
    shuffle=False)

    with torch.no_grad():                                    # warm-up (not
    timed): avoids counting one-off GPU start-up cost
        first_batch = next(iter(loader))
        with torch.autocast(device_type=DEVICE.type, enabled=info["use_amp"]):
            forward_batch(model, first_batch, info["kind"])

    test_loss, preds, labels, inference_time = predict(model, loader,
    info["kind"], info["use_amp"])
    label_ids = list(range(NUM_CLASSES))                     # fixed order:
    World, Sports, Business, Sci/Tech
    result = {
        "model": name,
        "test_loss": float(test_loss),
        "accuracy": float(accuracy_score(labels, preds)),
        "macro_precision": float(precision_score(labels, preds,
    average="macro", zero_division=0)),
        "macro_recall": float(recall_score(labels, preds, average="macro",
    zero_division=0)),
        "macro_f1": float(f1_score(labels, preds, average="macro",
    zero_division=0)),
        "train_time_sec": float(info["train_time_sec"]),
        "inference_time_sec": float(inference_time),
        "inference_ms_per_article": float(inference_time / len(labels) *
    1000),
        "params_total": int(info["params_total"]),
```

```
        "params_trainable": int(info["params_trainable"]),
        "best_epoch": int(info["best_epoch"]),
        "history": info["history"],
        "confusion_matrix": confusion_matrix(labels, preds,
    labels=label_ids).tolist(),
        "classification_report": classification_report(labels, preds,
    labels=label_ids, target_names=CLASS_NAMES,
                                                       output_dict=True,
    zero_division=0),
        "run_mode": "full" if USE_FULL_DATASET else "debug",
        "n_train": len(train_df), "n_val": len(val_df), "n_test":
    len(test_df),
    }
    with open(os.path.join(RESULTS_DIR, f"{slug(name)}_results.json"), "w") as
    f:
        json.dump(result, f, indent=2)
    print(f"\n===== {name} | test accuracy {result['accuracy']:.4f} | macro F1
    {result['macro_f1']:.4f} =====")
    print(classification_report(labels, preds, labels=label_ids,
    target_names=CLASS_NAMES, digits=4, zero_division=0))
    del model
    gc.collect()
    if DEVICE.type == "cuda":
        torch.cuda.empty_cache()

for name in MODEL_NAMES:
    evaluate_on_test(name)

# Reload everything from the saved JSON files: this is the structured record
    used by all later sections.
RESULTS = {}
for name in MODEL_NAMES:
    with open(os.path.join(RESULTS_DIR, f"{slug(name)}_results.json")) as f:
        RESULTS[name] = json.load(f)
print("\nSaved result files:", sorted(os.listdir(RESULTS_DIR)))
```

A.20 Confusion-matrix visualisation

Code (notebook code cell 20)

```
fig, axes = plt.subplots(2, 3, figsize=(17, 10))
for ax, name in zip(axes.ravel(), MODEL_NAMES):
    cm = np.array(RESULTS[name]["confusion_matrix"])
    ConfusionMatrixDisplay(cm, display_labels=CLASS_NAMES).plot(ax=ax,
    cmap="Blues", colorbar=False, xticks_rotation=30, values_format="d")
    ax.set_title(f"{name} (accuracy {RESULTS[name]['accuracy']:.4f})")
    ax.set_xlabel("Predicted class")
    ax.set_ylabel("True class")
axes.ravel()[-1].axis("off")
plt.tight_layout()
plt.show()

print("Automatic reading of the matrices:")
for name in MODEL_NAMES:
    cm = np.array(RESULTS[name]["confusion_matrix"])
    off_diag = cm.copy()
    np.fill_diagonal(off_diag, 0)
    recall_per_class = np.diag(cm) / cm.sum(axis=1)
    weakest = int(recall_per_class.argmin())
    if off_diag.max() == 0:
        mistake = "no mistakes"
```

```
    else:
        i, j = np.unravel_index(off_diag.argmax(), off_diag.shape)
        mistake = f"most frequent mistake: true '{CLASS_NAMES[i]}' predicted
    as '{CLASS_NAMES[j]}' ({off_diag[i, j]} articles)"
    print(f"- {name}: {mistake}; weakest class: {CLASS_NAMES[weakest]} (recall
    {recall_per_class[weakest]:.3f})")
```

A.21 Final model comparison table

Code (notebook code cell 21)

```
rows = []
for name in MODEL_NAMES:
    r = RESULTS[name]
    rows.append({
        "Model": name,
        "Accuracy": r["accuracy"],
        "Macro Precision": r["macro_precision"],
        "Macro Recall": r["macro_recall"],
        "Macro F1-score": r["macro_f1"],
        "Training Time (s)": r["train_time_sec"],
        "Inference Time (s)": r["inference_time_sec"],
        "Number of Parameters": r["params_total"],
    })
final_df = pd.DataFrame(rows).sort_values("Accuracy",
    ascending=False).reset_index(drop=True)
final_df.to_csv(os.path.join(RESULTS_DIR, "final_comparison.csv"),
    index=False)

display(final_df.style.format({"Accuracy": "{:.4f}", "Macro Precision":
    "{:.4f}", "Macro Recall": "{:.4f}",
                                "Macro F1-score": "{:.4f}", "Training Time
    (s)": "{:.1f}",
                                "Inference Time (s)": "{:.2f}", "Number of
    Parameters": "{:,}"}))
print("Run mode:", RESULTS[MODEL_NAMES[0]]["run_mode"], "| parameters include
    embeddings and pretrained weights for every model.")

colors = ["#4C78A8", "#F58518", "#54A24B", "#E45756", "#B279A2"]
def bar_panel(ax, column, title, ylabel, zoom=False, fmt="{:.4f}"):
    values = final_df[column].values
    bars = ax.bar(final_df["Model"], values,
    color=[colors[MODEL_NAMES.index(m)] for m in final_df["Model"]])
    if zoom:
        ax.set_ylim(max(0, values.min() - 0.05), min(1.0, values.max() +
    0.02))
    ax.set_title(title); ax.set_ylabel(ylabel); ax.set_xlabel("Model")
    ax.tick_params(axis="x", rotation=20)
    for b, v in zip(bars, values):
        ax.text(b.get_x() + b.get_width() / 2, b.get_height(), fmt.format(v),
    ha="center", va="bottom", fontsize=9)

fig, axes = plt.subplots(2, 2, figsize=(14, 9))
bar_panel(axes[0, 0], "Accuracy", "Test accuracy (zoomed y-axis)", "Accuracy",
    zoom=True)
bar_panel(axes[0, 1], "Macro F1-score", "Test macro F1-score (zoomed y-axis)",
    "Macro F1", zoom=True)
bar_panel(axes[1, 0], "Training Time (s)", "Training time", "Seconds",
    fmt="{:.1f}")
bar_panel(axes[1, 1], "Inference Time (s)", "Inference time on the test set",
    "Seconds", fmt="{:.2f}")
```

```
plt.tight_layout()
plt.show()
```

A.22 Comparative analysis of model performance

Code (notebook code cell 22)

```
def cmp_word(a, b):
    return "higher" if a > b else ("lower" if a < b else "equal")

get = lambda m: final_df[final_df["Model"] == m].iloc[0]
recurrent = final_df[final_df["Model"].isin(["Simple RNN", "LSTM", "GRU"])]
transformers_df = final_df[final_df["Model"].isin(["BERT", "DistilBERT"])]
best_overall = final_df.iloc[0]
best_rec, best_tr = recurrent.iloc[0], transformers_df.iloc[0]

print(f"1. Best model on the test set: {best_overall['Model']} (accuracy
    {best_overall['Accuracy']:.4f}, macro F1 {best_overall['Macro F1-
    score']:.4f}).")
gap_pp = (best_tr["Accuracy"] - best_rec["Accuracy"]) * 100
print(f"2. Best Transformer ({best_tr['Model']}) vs best recurrent model
    ({best_rec['Model']}): accuracy is {cmp_word(best_tr['Accuracy'],
    best_rec['Accuracy'])} "
      f"by {abs(gap_pp):.2f} percentage points.")
print(f"   Cost of that comparison: training time x{best_tr['Training Time
    (s)'] / best_rec['Training Time (s)']:.1f}, "
      f"inference time x{best_tr['Inference Time (s)'] / best_rec['Inference
    Time (s)']:.1f}, "
      f"parameters x{best_tr['Number of Parameters'] / best_rec['Number of
    Parameters']:.1f} (Transformer relative to recurrent).")
rnn_acc = get("Simple RNN")["Accuracy"]
for m in ["LSTM", "GRU"]:
    d = (get(m)["Accuracy"] - rnn_acc) * 100
    print(f"3. {m} accuracy is {cmp_word(get(m)['Accuracy'], rnn_acc)} than
    Simple RNN by {abs(d):.2f} percentage points.")
b, d_ = get("BERT"), get("DistilBERT")
print(f"4. DistilBERT vs BERT: accuracy {cmp_word(d_['Accuracy'],
    b['Accuracy'])} by {abs(d_['Accuracy'] - b['Accuracy']) * 100:.2f}
    percentage points; "
      f"training time x{d_['Training Time (s)'] / b['Training Time (s)']:.2f},
    inference time x{d_['Inference Time (s)'] / b['Inference Time (s)']:.2f}, "
      f"parameters x{d_['Number of Parameters'] / b['Number of
    Parameters']:.2f} (DistilBERT relative to BERT).")
print("5. Ranking by accuracy per training-second (higher = cheaper
    accuracy):")
eff = final_df.assign(AccuracyPerTrainingSecond=final_df["Accuracy"] /
    final_df["Training Time (s)"]).sort_values("AccuracyPerTrainingSecond",
    ascending=False)
display(eff[["Model", "Accuracy", "Training Time (s)",
    "AccuracyPerTrainingSecond"]].round(5))
if RESULTS[MODEL_NAMES[0]]["run_mode"] == "debug":
    print("\nCAUTION: these numbers come from the small debug subset and are
    only a sanity check, not final findings.")
print("Reminder: one seed was used, so differences of a few tenths of a
    percentage point may be within random variation.")
```

A.23 Overall results summary and conclusion

Code (notebook code cell 23)

```
mode_text = "the full AG News dataset" if RESULTS[MODEL_NAMES[0]]["run_mode"]
    == "full" else "a small debug subset (not final)"
```

```
best = final_df.iloc[0]
rank_text = ", ".join(f"{r.Model} ({r.Accuracy:.4f})" for r in
    final_df.itertuples())

summary = f"""
### Experiment summary
Five models (Simple RNN, LSTM, GRU, BERT, DistilBERT) were trained on
    {mode_text}, using {RESULTS[MODEL_NAMES[0]]['n_train']:,} training,
    {RESULTS[MODEL_NAMES[0]]['n_val']:,} validation and
    {RESULTS[MODEL_NAMES[0]]['n_test']:,} test articles with identical splits.
Test accuracy ranking: {rank_text}.

### Best model
**{best['Model']}** achieved the highest test accuracy
    ({best['Accuracy']:.4f}) and macro F1-score ({best['Macro F1-score']:.4f})
    among the models ranked by accuracy.
It needed {best['Training Time (s)']:.1f} s of training, {best['Inference Time
    (s)']:.2f} s to classify the test set and has {best['Number of
    Parameters']:,} parameters.

### Research questions
- **RQ1 (Transformers vs recurrent):** best Transformer accuracy =
    {transformers_df.iloc[0]['Accuracy']:.4f}
    ({transformers_df.iloc[0]['Model']}); best recurrent accuracy =
    {recurrent.iloc[0]['Accuracy']:.4f} ({recurrent.iloc[0]['Model']}).
- **RQ2 (gating):** Simple RNN {get('Simple RNN')['Accuracy']:.4f}, LSTM
    {get('LSTM')['Accuracy']:.4f}, GRU {get('GRU')['Accuracy']:.4f}.
- **RQ3 (cost):** see the training time, inference time and parameter columns
    of the comparison table; Section 15 reports the exact ratios.
- **RQ4 (DistilBERT vs BERT):** DistilBERT {get('DistilBERT')['Accuracy']:.4f}
    vs BERT {get('BERT')['Accuracy']:.4f} accuracy, with DistilBERT using
    {get('DistilBERT')['Number of Parameters'] / get('BERT')['Number of
    Parameters'] * 100:.0f}% of BERT's parameters.
"""
display(Markdown(summary))
```